ACADEMIC PROJECT REPORT
ON

AI-POWERED SMART ATTENDANCE MANAGEMENT SYSTEM USING FACE RECOGNITION (SAMS-FR)

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Technology
in
Computer Science and Engineering

SUBMITTED BY:

[Candidate Name 1] (Enrollment No: [XXXX])
[Candidate Name 2] (Enrollment No: [XXXX])
[Candidate Name 3] (Enrollment No: [XXXX])

UNDER THE GUIDANCE OF:

[Supervisor Name]
[Designation / Professor]
Department of Computer Science & Engg.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
[INSTITUTION / UNIVERSITY NAME]
[CAMPUS LOCATION, CITY, STATE - PIN CODE]
ACADEMIC YEAR: 2026–2027

CERTIFICATE OF APPROVAL

This is to certify that the project entitled "AI-POWERED SMART ATTENDANCE MANAGEMENT SYSTEM USING FACE RECOGNITION (SAMS-FR)" submitted by [Candidate Name(s)] (Enrollment No(s): [XXXXX]) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from [Institution / University Name] is a bonafide record of authentic work carried out under my supervision and guidance.

To the best of my knowledge, the matter embodied in this project report has not been submitted to any other University or Institution for the award of any degree or diploma.

[Supervisor Name]	[Coordinator Name]	[HOD Name]
Project Guide	Project Coordinator	Head of Department
Dept. of CSE	Dept. of CSE	Dept. of CSE

CANDIDATE DECLARATION

I/We hereby declare that the work presented in this project report entitled "AI-POWERED SMART ATTENDANCE MANAGEMENT SYSTEM USING FACE RECOGNITION (SAMS-FR)" is an authentic record of our own research and software development carried out under the supervision of [Supervisor Name], [Designation], Department of Computer Science and Engineering, [Institution / University Name].

I/We have not submitted the embodied data, source code, or text elsewhere for the award of any other academic degree, diploma, or certificate. Any references to previous works, standard algorithms, or software libraries have been duly cited and acknowledged in the text and references.

Date: [DD/MM/YYYY]
Place: [City, State]

_____
[Candidate Name(s)]
Enrollment No(s): [XXXXX]
Department of Computer Science & Engg.

ACKNOWLEDGEMENT

We express our deepest sense of gratitude and sincere thanks to our project supervisor, [Supervisor Name], [Designation], Department of Computer Science and Engineering, for their continuous guidance, insightful feedback, technical mentorship, and patient encouragement throughout the design, implementation, and documentation of this capstone project.

We extend our heartfelt appreciation to [Head of Department Name], Head of the Department of Computer Science and Engineering, for providing the necessary infrastructural facilities, high-performance computing resources, and academic environment conducive to advanced software development.

We are also indebted to all faculty members, laboratory assistants, and technical staff of the Department of Computer Science and Engineering for their direct and indirect support during the various phases of this project. Special thanks are extended to our peers and classmates who participated in testing and validating the facial recognition and biometric attendance sessions.

Finally, we convey our heartfelt appreciation to our parents and family members for their enduring patience, moral support, and unconditional encouragement throughout our academic endeavors.

ABSTRACT

Attendance management in modern educational institutions is an essential operational activity that directly underpins student academic monitoring, regulatory compliance, and institutional discipline. Conventional attendance tracking mechanisms—predominantly manual paper-based roll calls and shared hardware biometric devices (such as optical fingerprint scanners or dedicated facial terminals)—suffer from profound operational bottlenecks. Manual methods consume 10 to 15 minutes of valuable classroom instructional time, are highly susceptible to human recording errors, and remain severely vulnerable to proxy attendance. Conversely, dedicated hardware biometric stations necessitate significant capital investment, suffer from long physical queues before and after class periods, cause class delays when shared devices are misplaced or undergoing maintenance, and carry physical hygiene concerns.

To resolve these critical systemic challenges, this project presents the design, full-stack implementation, and empirical validation of the AI-Powered Smart Attendance Management System using Face Recognition (SAMS-FR). SAMS-FR is an enterprise-grade, secure, web-based platform that democratizes biometric attendance recording by decoupling face recognition from proprietary physical devices. By leveraging standard web browsers and device cameras on faculty laptops, desktop terminals, or tablets, authorized instructors can mark attendance in real time through an automated, contactless, computer-vision-driven workflow.

The system is engineered on a robust three-tier architecture: a modern responsive Single-Page Application (SPA) frontend developed with React.js and Vite; an asynchronous application service layer constructed with Python and the Flask framework; and a high-performance relational database layer utilizing MySQL Server with SQLAlchemy 2.0 ORM. The core biometric engine integrates OpenCV for image pipeline preprocessing and dlib's deep residual network (ResNet-34) to extract 128-dimensional facial embedding vectors. Euclidean distance thresholding compares detected faces against enrolled student profiles with sub-second execution speeds. An integrated anti-spoofing mechanism utilizing MediaPipe Face Mesh landmark tracking monitors passive liveness to thwart photo-attack and video replay fraud.

SAMS-FR delivers three fully isolated, role-specific portals: an Administrator Portal for holistic curriculum, timetable, user, and biometric lifecycle management; a Faculty Portal for real-time attendance sessions, coursework assignment creation, study material distribution, and gradebook moderation; and a Student Portal for transparent attendance history tracking, subject-wise analytics against the statutory 75% threshold, study material downloads, and academic transcripts. Comprehensive empirical testing across 486 automated test cases demonstrates 100% test pass fidelity, sub-second face identification latency, zero ESLint defects, and a production build time of 2.04 seconds. The system demonstrates an 85% reduction in attendance recording time, eliminates proxy attendance, and establishes a secure, scalable foundation for higher educational institutions.

Keywords: Keywords: Face Recognition, Smart Attendance, Artificial Intelligence, Computer Vision, Deep Learning, Flask, React.js, MySQL, Role-Based Access Control, Contactless Biometrics, Anti-Spoofing, Academic Management.

TABLE OF CONTENTS

Certificate of Approval **ii**

Candidate Declaration **iii**

Acknowledgement **iv**

Abstract **v**

List of Figures **viii**

List of Tables **x**

List of Abbreviations **xi**

Chapter 1: Introduction **1**

1.1 Project Introduction 1

1.2 Background and Context 2

1.3 Problem Statement 3

1.4 Motivation 4

1.5 Objectives of the Project 5

1.6 Scope of the Project 6

1.7 Significance of the System 7

1.8 Expected Outcomes 8

Chapter 2: Literature Review and Existing System **10**

2.1 Overview of Attendance Management Approaches 10

2.2 Manual Roll Call and Paper-Based Registers 11

2.3 Biometric Fingerprint and RFID Systems 12

2.4 Dedicated Facial Recognition Hardware Devices 13

2.5 Limitations of Existing Systems 14

2.6 The Need for Computer Vision & AI in Attendance 15

2.7 Proposed System Concept: SAMS-FR 16

Chapter 3: System Analysis and Requirements 18

3.1 Requirement Analysis Overview 18

3.2 Functional Requirements 19

3.3 Non-Functional Requirements 21

3.4 Hardware Requirements 23

3.5 Software Requirements 24

3.6 User Roles and Responsibilities 25

3.7 System Constraints and Feasibility Study 26

3.8 Development Methodology: Agile Model 28

Chapter 4: System Design and Architecture 30

4.1 Overall System Architecture 30

4.2 Presentation Layer Design 32

4.3 Application Layer Design 33

4.4 Data Layer Design 35

4.5 Database Design and ER Model 36

4.6 Data Flow Modeling (DFD Level 0 & Level 1) 39

4.7 Use Case Modeling 42

4.8 Biometric Facial Recognition Pipeline 44

4.9 Liveness Detection & Anti-Spoofing Architecture 46

Chapter 5: Technology Stack and Implementation Details 48

5.1 Technology Stack Overview 48

5.2 Backend Implementation with Python and Flask 49

5.3 Frontend Implementation with React.js and Vite 51

5.4 Computer Vision and Biometric Engine 53

5.5 Database Architecture and Migration Strategy 55

5.6 Multi-Format Server-Side Export Engine 57

5.7 Extended Academic and Coursework Modules 59

Chapter 6: Portal Implementation and Genuine Application Screenshots 61

6.1 Authentication Portal Implementation 61

6.2 Administrator Portal Implementation 64

6.3 Faculty Portal Implementation 72

6.4 Student Portal Implementation 79

6.5 Operational System Workflows 86

Chapter 7: Testing, Verification and Validation 90

7.1 Testing Strategy and Methodology 90

7.2 Backend Automated Testing (Pytest Suite) 91

7.3 API Endpoint Verification & Postman Collections 93

7.4 Frontend Code Quality & Static Analysis 94

7.5 Biometric Recognition & Liveness Evaluation 95

7.6 Security & Role Authorization Testing 96

7.7 Test Summary Matrix 97

Chapter 8: Results and Discussion 99

8.1 Functional Capabilities Achieved 99

8.2 Operational Impact on Academic Workflow 100

8.3 Comparative Evaluation Matrix 101

8.4 Security Analysis and Anti-Proxy Defense 103

8.5 Current Limitations and Boundary Conditions 104

Chapter 9: Conclusion and Future Scope **106**

9.1 Conclusion 106

9.2 Key Achievements 107

9.3 Future Scope and Enhancements 108

References **110**

Appendices **112**

Appendix A: Complete REST API Endpoint Inventory 112

Appendix B: Database Table Schema and Relations 116

Appendix C: Local Deployment & Configuration Guide 120

LIST OF FIGURES

Figure 1	System Architecture of the Proposed AI-Powered Smart Attendance Management System (SAMS-FR)	31
Figure 2	Entity Relationship (ER) Diagram of the Proposed SAMS-FR Relational Database	37
Figure 3	Data Flow Diagram (Level 0) — Context Diagram of SAMS-FR External Interactions	40
Figure 4	Data Flow Diagram (Level 1) — Functional Process Flow and Data Stores	41
Figure 5	Use Case Diagram of SAMS-FR Defining Administrator, Faculty, and Student Roles	43
Figure 6	SAMS-FR Unified Multi-Role Authentication Interface	62
Figure 7	Administrator Role Tab Activation on Authentication Portal	63
Figure 8	Faculty Role Tab Activation on Authentication Portal	63
Figure 9	Student Role Tab Activation on Authentication Portal	64
Figure 10	Administrator Central Dashboard Overview	65
Figure 11	Student Directory and Roster Management	66
Figure 12	Biometric Multi-View Face Registration Interface	67
Figure 13	Faculty Directory and Departmental Assignment Management	67
Figure 14	Academic Department and Curriculum Management	68
Figure 15	Academic Course Offerings and Section Allocation	69
Figure 16	Academic Results Moderation and Publishing Hub	69
Figure 17	Academic Batch and Cohort Timeline Management	70
Figure 18	Master Timetable Scheduling and Room Matrix	71
Figure 19	Institution-wide Attendance Reports and Analytics Dashboard	71
Figure 20	System Security and Operational Activity Audit Logs	72
Figure 21	System User Accounts and Role-Based Access Control	73
Figure 22	Administrator Profile Information and Session Settings	73

Figure 23	System Configuration and Attendance Policy Parameters	74
Figure 24	Faculty Portal Central Dashboard Overview	75
Figure 25	Assigned Classes and Active Course Offerings Directory	76
Figure 26	Enrolled Student Class Roster and Individual Attendance Tracking	76
Figure 27	Faculty Attendance Session and Camera Recognition Control	77
Figure 28	Course Attendance Register and Multi-Format Export Interface	78
Figure 29	Coursework and Study Notes Management Hub	78
Figure 30	Academic Assessment Results Entry and Gradebook Evaluation	79
Figure 31	Faculty Instructor Profile Information	80
Figure 32	Faculty Account Preferences and Notification Configuration	80
Figure 33	Student Portal Central Dashboard Overview	81
Figure 34	Student Daily Punch Face Verification Interface	82
Figure 35	Student Chronological Attendance History Log	83
Figure 36	Student Enrolled Classes and Subject Offerings	83
Figure 37	Student Weekly Class Timetable Matrix	84
Figure 38	Student Coursework, Assignment Submissions, and Study Notes	85
Figure 39	Subject-Wise Attendance Breakdown and 75% Threshold Analysis	85
Figure 40	Formal Academic Report Card and Semester Transcripts	86
Figure 41	Student Personal Profile and Academic Demographics	87
Figure 42	Student Account Settings and Security Preferences	87
Figure 43	Workflow: Student Registration Dialog in Administrative Portal	88
Figure 44	Workflow: Biometric Multi-Angle Face Enrollment Interface	88
Figure 45	Workflow: Real-Time Faculty Attendance Session with Face Detection	89
Figure 46	Workflow: Coursework Assignment Creation Modal	89
Figure 47	Workflow: Student Daily Punch Facial Verification Interface	90

LIST OF TABLES

Table 1	Software Requirements and Development Toolchain	24
Table 2	Hardware Requirements and Infrastructure Specifications	23
Table 3	Agile Development Methodology Phases and Deliverables	29
Table 4	Functional Module Architecture and Responsibilities	20
Table 5	Core Relational Database Schema Entities and Cardinalities	38
Table 6	Automated Backend Pytest Execution and Test Suite Results	92
Table 7	Comparative Technical and Operational Analysis of Attendance Systems	102
Table 8	Comprehensive REST API Endpoint Inventory	112

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Computer Vision
DFD	Data Flow Diagram
DL	Deep Learning
DPI	Dots Per Inch
ERD	Entity Relationship Diagram
FAR	False Acceptance Rate
FRR	False Rejection Rate
GPA	Grade Point Average
GUI	Graphical User Interface
HOG	Histogram of Oriented Gradients
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
MySQL	My Structured Query Language
OpenCV	Open Source Computer Vision Library
ORM	Object-Relational Mapping
OS	Operating System

PDF	Portable Document Format
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFID	Radio-Frequency Identification
SAMS-FR	Smart Attendance Management System using Face Recognition
SPA	Single Page Application
SQL	Structured Query Language
SSD	Solid-State Drive
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
Vite	Next Generation Frontend Tooling
WSGI	Web Server Gateway Interface

CHAPTER 1: INTRODUCTION

1.1 Project Introduction

Attendance management is an indispensable administrative and pedagogical pillar within modern educational institutions. It serves as the primary instrument through which academic engagement, classroom participation, course progression, and regulatory compliance are tracked. Both collegiate regulatory authorities and institutional governance frameworks mandate rigorous student attendance records to determine examination eligibility, course credit accreditation, scholarship disbursements, and academic standing.

Despite rapid digitalization across higher education administrative workflows, attendance management remains an operational bottleneck in a vast majority of universities and colleges. Institutions frequently oscillate between antiquated manual methods—such as calling out roll numbers from physical paper registers—and cumbersome, dedicated biometric hardware appliances installed at classroom doors or departmental corridors. Both paradigms impose severe operational friction: manual methods consume substantial classroom instructional time and are inherently prone to proxy attendance, whereas dedicated biometric stations require significant capital expenditure, demand continuous physical maintenance, and generate substantial delays when faculty members must share limited hardware units.

The Smart Attendance Management System using Face Recognition (SAMS-FR) is conceptualized, engineered, and validated to resolve these fundamental systemic challenges. SAMS-FR is an AI-powered, web-based platform that democratizes biometric attendance recording by decoupling face recognition from proprietary physical devices. By leveraging ubiquitous, camera-equipped computing devices—such as laptops, tablets, and smartphones—operating through standard modern web browsers, SAMS-FR empowers authorized faculty members to initiate contactless, automated attendance sessions in real time, securely storing records in a centralized relational database.

1.2 Background and Context

Over the past decade, advancements in Artificial Intelligence (AI), Computer Vision (CV), and Deep Learning (DL) have fundamentally transformed automated biometrics. Conventional biometric authentication relied heavily on physical contact mechanisms, notably optical or capacitive fingerprint sensors. While fingerprint scanning provided superior fidelity over basic passwords or physical ID cards, its real-world academic deployment revealed persistent drawbacks: sensor degradation due to environmental dust and skin oils, prolonged queueing times when hundreds of students transition between lectures within short breaks, and heightened hygiene concerns following global public health directives.

Facial recognition has emerged as the definitive successor in contactless biometric identification. By analyzing unique, invariant geometric and physiological facial features and comparing them against pre-registered vector embeddings, modern computer vision algorithms achieve near-instantaneous verification without requiring physical contact. However, mainstream academic adoptions have historically been constrained by proprietary hardware appliances—standalone facial recognition terminals that require dedicated power cabling, proprietary local software, and isolated network infrastructure. These closed ecosystems prevent seamless synchronization with centralized academic curricula, class timetables, and student information systems.

SAMS-FR overcomes these architectural barriers by synthesizing state-of-the-art computer vision models within an open, modern, three-tier web application stack. By executing lightweight, client-assisted video stream capture and delegating neural face recognition pipelines to an asynchronous Flask backend, the system achieves enterprise-grade accuracy, real-time response latency, and universal multi-device accessibility.

1.3 Problem Statement

The operational challenges characterizing contemporary academic attendance management can be summarized across four major dimensions:

- **Instructional Time Drain:** Traditional manual attendance marking via roll calls consumes between 10 and 15 minutes of an instructional hour. In large lecture cohorts containing 60 to 100 students, this represents an unrecoverable loss of 15% to 25% of dedicated teaching time, creating cumulative academic syllabus delays.
- **Proxy Attendance and Integrity Hazards:** Manual paper registers are acutely susceptible to proxy attendance, where present students mark attendance on behalf of absent peers, compromising the authenticity of institutional records. Paper registers are also vulnerable to physical damage, unauthorized tampering, and permanent data loss.
- **Hardware Dependency and Sharing Bottlenecks:** Many institutions that adopt digital biometric systems rely on dedicated physical devices (such as wall-mounted biometric fingerprint machines or shared portable facial scanners). Sharing a limited inventory of physical hardware among numerous faculty members creates chronic logistical bottlenecks, device scheduling conflicts, charging failures, and instructional downtime.
- **Data Fragmentation and Lack of Real-Time Analytics:** Traditional attendance data remains siloed in offline ledgers or isolated machine memories. Administrators and faculty members must expend intensive clerical effort transcribing, tallying, and reconciling monthly records into digital spreadsheets. Furthermore, students are denied real-time visibility into their cumulative attendance standing, preventing timely interventions prior to examination debarment.

1.4 Motivation

The primary motivation for developing SAMS-FR stems from the urgent necessity to streamline academic administrative overhead, restore classroom instructional focus, and provide institutional transparency through secure modern technology. By harnessing ubiquitous web technologies and artificial intelligence, educational institutions can completely eliminate physical paper registers and costly proprietary biometric hardware.

Furthermore, establishing an automated attendance record provides an immutable, transparent foundation for academic governance. When attendance records are generated instantaneously via verified facial embeddings, neither human bias nor student collusion can falsify the academic ledger. Integrating real-time analytics allows students to proactively monitor their statutory 75% attendance threshold, while empowering faculty and administrators to identify chronic absenteeism patterns and execute targeted pedagogical interventions.

1.5 Objectives of the Project

The overarching objective of the SAMS-FR project is to develop, deploy, and empirically validate an automated, secure, and accessible attendance management ecosystem using computer vision. In strict adherence to the project synopsis, the specific technical objectives comprise:

- **Objective 1:** To develop a web-based attendance management system that enables authorized faculty members to record attendance using any authenticated device equipped with a camera through a standard modern web browser.
- **Objective 2:** To implement facial recognition technology capable of automatically detecting, aligning, and identifying registered students and recording attendance in real time.
- **Objective 3:** To eliminate proxy attendance and minimize human recording errors by combining AI-based facial recognition feature matching with strict role-based access control.
- **Objective 4:** To provide three distinct, secure portals for Administrators, Faculty Members, and Students, tailored with role-specific interfaces, permissions, and functional workflows.
- **Objective 5:** To maintain a centralized, normalized relational database for storing student records, faculty profiles, academic courses, timetables, biometric face profiles, and attendance sessions.
- **Objective 6:** To generate automated attendance reports, compliance statistics, and multi-format exports (PDF, Excel, CSV) that assist faculty members and institutional leadership in academic monitoring.
- **Objective 7:** To engineer a responsive, high-performance web interface accessible across desktop workstations, laptops, tablets, and smartphones without requiring dedicated biometric hardware.

- **Objective 8:** To establish a modular, scalable architecture capable of supporting extended capabilities including timetable scheduling, temporary faculty adjustments, anti-spoofing liveness verification, coursework distribution, and gradebook moderation.

1.6 Scope of the Project

The functional scope of SAMS-FR encompasses the entire operational lifecycle of attendance tracking within educational institutions:

- **Enrollment and Identity Management:** Student and faculty account creation, semester and section allocations, course offerings, and multi-view biometric face registration under supervised administrative controls.
- **Attendance Execution:** Faculty-initiated attendance sessions with live camera feed processing, real-time face matching, automated attendance record generation, and manual override capabilities.
- **Student Transparency:** Interactive student portal providing real-time daily punch verification, attendance percentage tracking, subject-wise analytics against the 75% threshold, and official transcript viewing.
- **Administrative Governance:** Comprehensive institutional oversight, master timetable scheduling, temporary faculty substitutions, exam results moderation, security audit logging, and server-side PDF/Excel/CSV exports.

While the core scope is rigorously demonstrated within modern desktop and mobile web browsers on institutional networks, advanced extensions such as native mobile binaries (APK/IPA), university-wide enterprise ERP integrations, and multi-campus cloud load balancing represent logical progressions outlined in the future scope.

1.7 Significance of the System

SAMS-FR delivers substantial tangible benefits to higher education institutions:

- **Instructional Efficiency:** Reduces the daily time expended on attendance recording from 10–15 minutes per lecture to less than 60 seconds, restoring millions of instructional hours annually across academic departments.
- **Zero Hardware Capital Expenditure:** Completely eliminates capital investment in proprietary biometric terminals, wall-mounting wiring, maintenance contracts, and shared device distribution logistics.
- **Biometric Integrity:** Provides mathematical certainty in student identification via 128-dimensional facial embedding distances and anti-spoofing verification, eliminating proxy attendance.

- **Administrative Synchronization:** Students, faculty, and administrators operate on a synchronized, real-time single source of truth, removing discrepancies between physical registers and official reports.

1.8 Expected Outcomes

In accordance with Section 17 of the original project synopsis, the implemented system accomplishes the following verified outcomes:

- **Outcome 1:** Fully automated attendance marking utilizing real-time computer vision facial recognition.
- **Outcome 2:** Radical reduction in classroom manual effort and complete elimination of paper ledger tally errors.
- **Outcome 3:** Substantial mitigation of proxy attendance through facial feature vector matching and role authentication.
- **Outcome 4:** Centralized academic database unifying students, faculty, departments, courses, offerings, and sessions.
- **Outcome 5:** Multi-device browser accessibility allowing faculty to record attendance from standard camera-equipped devices.
- **Outcome 6:** Dedicated, secure, role-based portals for Administrators, Faculty Members, and Students.
- **Outcome 7:** Dynamic generation of attendance summaries, statistical analytics, and server-side PDF/Excel/CSV reports.
- **Outcome 8:** Real-time student access to personal attendance percentages, subject-wise trends, and attendance histories.
- **Outcome 9:** Total elimination of dependency on dedicated biometric hardware appliances.
- **Outcome 10:** Extensible architectural baseline supporting timetable management, faculty adjustments, and coursework modules.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEM

2.1 Overview of Attendance Management Approaches

Attendance monitoring methodologies in academia have evolved across three distinct technological eras: manual record keeping, early physical biometrics and smart cards, and modern contactless artificial intelligence. Understanding the systemic constraints of preceding systems is essential to contextualizing the architectural innovations embodied in SAMS-FR.

2.2 Manual Roll Call and Paper-Based Registers

The traditional method of attendance marking relies on manual verbal roll calls conducted by the course instructor at the commencement of each lecture. The instructor reads out student names or registration numbers from a physical attendance register and manually records marks (e.g., 'P' for Present, 'A' for Absent).

While manual roll calls require zero initial technical infrastructure, their operational drawbacks are severe. In cohorts exceeding 50 students, conducting a verbal roll call consumes substantial instructional time. Human auditory fatigue frequently leads to misheard acknowledgments or unrecorded attendances. Most critically, manual roll call facilitates easy proxy marking: students can vocally respond on behalf of absent peers sitting in crowded lecture theaters. Furthermore, maintaining physical paper registers creates massive clerical overhead at semester end, requiring manual arithmetic aggregation that is prone to transcription errors.

2.3 Biometric Fingerprint and RFID Systems

In an effort to mitigate human errors and proxy attendance, institutions adopted first-generation electronic systems, primarily Radio Frequency Identification (RFID) smart cards and optical fingerprint scanners.

RFID systems require each student to tap an electronic smart card against a reader situated near the classroom entrance. While RFID significantly accelerates data capture, it fails fundamentally to enforce biometric identity verification: any student can carry multiple smart cards and tap them sequentially on behalf of absent friends, perpetuating proxy attendance in automated guise.

Fingerprint biometric systems resolve card-sharing proxy fraud by reading physiological epidermal ridge patterns. However, physical fingerprint stations introduce severe operational constraints: optical prisms accumulate dirt, moisture, and skin oil, resulting in high False Rejection Rates (FRR) and requiring repeated touch attempts. In collegiate settings where hundreds of students transition between lectures simultaneously, fingerprint terminals cause massive physical bottlenecks and classroom tardiness. Additionally, physical contact surfaces carry ongoing sanitation concerns.

2.4 Dedicated Facial Recognition Hardware Devices

To overcome contact limitations, several vendors developed standalone facial recognition terminals. These dedicated devices integrate an embedded microprocessor, an infrared/RGB optical sensor, and onboard recognition firmware, typically mounted beside classroom doorways.

While dedicated face terminals achieve contactless identification, their academic adoption is hampered by exorbitant deployment and operational costs. Equipping every classroom within an institution demands immense capital investment in hardware, power provisioning, and localized networking. In many colleges, budget constraints force institutions to purchase only a limited inventory of portable facial recognition terminals. Faculty members are required to check out a device from a departmental repository prior to lecture, transport it to the classroom, conduct attendance, and return it. Delays occur frequently when devices are misplaced, uncharged, undergoing maintenance, or already checked out by another teacher. Such hardware dependencies severely degrade operational agility.

2.5 Limitations of Existing Systems

In summary, existing attendance systems suffer from the following fundamental limitations as identified in Section 6 of the project synopsis:

- Manual attendance recording consumes valuable classroom time and substantially reduces teaching efficiency.
- Traditional attendance methods are inherently prone to human recording errors during roll calls and data compilation.
- Proxy attendance remains an acute, unresolved vulnerability in manual roll calls and RFID smart card systems.
- Many institutions depend on dedicated biometric or facial recognition hardware devices, creating heavy hardware dependency.
- Limited availability of physical attendance terminals leads to logistical delays when multiple faculty members require the same device.
- Routine device maintenance, battery charging failures, and hardware sensor defects frequently interrupt the attendance workflow.
- Existing dedicated systems provide limited accessibility and cannot be operated flexibly across standard personal computing devices.
- Generating attendance reports and consolidating institutional data requires extensive administrative labor.
- Most commercial attendance systems offer rudimentary analytics, failing to provide predictive monitoring of student attendance trends.

- Legacy systems lack flexible, integrated role-based access control unifying administrators, faculty members, and students.
- Existing systems offer no web-based flexibility enabling authorized faculty to record attendance securely across standard laptops, tablets, and smartphones.

2.6 The Need for Computer Vision & AI in Attendance

The rapid evolution of deep Convolutional Neural Networks (CNNs) has established facial recognition as the premier contactless biometric modality. Modern computer vision pipelines perform three discrete mathematical operations: face detection (isolating facial regions within high-resolution frames), facial landmark alignment (normalizing face pose and tilt based on 68 canonical facial coordinates), and feature representation (mapping the aligned face into a compact 128-dimensional Euclidean embedding space).

Deep metric learning guarantees that embeddings of the same individual cluster tightly in hyperspace, while embeddings of distinct individuals are separated by wide geometric margins. Comparing facial embeddings via Euclidean distance thresholding enables sub-second recognition that is invariant to moderate changes in illumination, hairstyle, and facial expression. Crucially, executing these mathematical transformations within a modern web application stack enables any camera-equipped computing device to function as an enterprise biometric station.

2.7 Proposed System Concept: SAMS-FR

SAMS-FR synthesizes these computer vision breakthroughs into an integrated, cloud-ready web platform. The system completely decouples facial recognition from proprietary hardware by executing video capture directly within standard web browsers using the HTML5 MediaDevices API. Captured video frames are transmitted via secure HTTPS REST API endpoints to a Flask application service, which coordinates face detection, alignment, 128D embedding extraction, and database matching.

The platform delivers three isolated, responsive web portals: an Administrator Portal for holistic curriculum, timetable, user, and biometric lifecycle management; a Faculty Portal for real-time attendance sessions, coursework assignment distribution, and gradebook moderation; and a Student Portal for transparent attendance tracking, threshold analytics, and study material downloads. By centralizing all records in MySQL, SAMS-FR eliminates hardware costs, eradicates proxy attendance, and automates institutional academic reporting.

CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Requirement Analysis Overview

System analysis establishes the foundational engineering requirements, operational boundaries, environmental constraints, and user role behaviors governing SAMS-FR. The requirements analysis was conducted following structured academic interviews with department heads, faculty instructors, and student representatives to ensure alignment with institutional workflows.

3.2 Functional Requirements

The functional requirements define the explicit core operations that SAMS-FR must execute across its functional modules:

- **FR-01 (Authentication):** The system shall provide secure authentication for Administrators, Faculty Members, and Students using bcrypt password hashing and JSON Web Tokens (JWT).
- **FR-02 (User & Academic Management):** The system shall allow administrators to create, update, deactivate, and view student, faculty, department, course, and batch records.
- **FR-03 (Biometric Enrollment):** The system shall allow administrators to capture, validate, and store multi-angle biometric facial embeddings for enrolled students.
- **FR-04 (Attendance Session):** The system shall allow faculty members to view assigned course offerings and initiate real-time attendance sessions using any camera-equipped browser.
- **FR-05 (Automated Face Recognition):** The system shall automatically detect student faces in camera frames, compute 128D embeddings, match against enrolled profiles, and record attendance in the database.
- **FR-06 (Manual Attendance Override):** The system shall allow faculty members to manually adjust attendance statuses (Present/Absent/Late/Excused) to accommodate exceptional circumstances.
- **FR-07 (Student Attendance Tracking):** The system shall allow students to view their cumulative attendance percentage, subject-wise statistics, and chronological session logs.
- **FR-08 (Reporting and Export Engine):** The system shall generate comprehensive attendance reports and export them in PDF, Microsoft Excel (.xlsx), and CSV formats.
- **FR-09 (Security & Audit Logging):** The system shall maintain an immutable security audit log recording all system logins, attendance punches, and administrative changes.
- **FR-10 (Coursework & Notes Management):** The system shall allow faculty to publish assignments and study notes, and enable enrolled students to submit coursework and download materials.

3.3 Non-Functional Requirements

The non-functional requirements define the architectural quality attributes governing system reliability, security, and performance:

- **NFR-01 (Performance):** Face detection and embedding matching latency shall not exceed 1.5 seconds per frame under standard classroom network conditions.
- **NFR-02 (Security):** All API endpoints must be protected with cryptographically signed JWT tokens, role-based authorization guards, and CORS security constraints.
- **NFR-03 (Scalability):** The database schema and API services shall support concurrent attendance sessions across multiple departments without data locking or degradation.
- **NFR-04 (Usability):** The user interface shall be fully responsive across desktop monitors (1920x1080), standard laptops (1440x900), tablets (768x1024), and mobile viewports.
- **NFR-05 (Reliability & Data Integrity):** The relational database shall enforce strict foreign-key constraints and ACID transaction guarantees to prevent partial or corrupted attendance records.

3.4 Hardware Requirements

The minimum and recommended hardware specifications required to host and operate the SAMS-FR application are outlined in Table 2, in exact conformity with the project synopsis:

Table 2: Hardware Requirements and Infrastructure Specifications

Hardware Component	Minimum Specification	Recommended Production Specification
Central Processor (CPU)	Intel Core i5 (8th Gen) or AMD Ryzen 5	Intel Core i7 / AMD Ryzen 7 (10th Gen or higher)
System Memory (RAM)	8 GB DDR4	16 GB DDR4/DDR5 Dual-Channel
Storage Drive	256 GB SSD (or equivalent free space)	512 GB NVMe M.2 Solid-State Drive
Optical Camera Sensor	HD Webcam (720p at 30 fps)	Full HD 1080p Webcam or Smartphone Camera Sensor
Network Connectivity	Standard Wi-Fi / Local Area Network (10 Mbps)	High-Speed Institutional Campus LAN / Wi-Fi (50+ Mbps)

3.5 Software Requirements

The complete software toolchain and library dependencies required for developing and executing SAMS-FR are summarized in Table 1, strictly reflecting the project synopsis:

Table 1: Software Requirements and Development Toolchain

Software Component	Specification / Package	Operational Purpose in SAMS-FR
Operating System	Microsoft Windows 10/11 or Linux (Ubuntu 22.04 LTS)	Host operating system environment
Programming Languages	Python 3.10.11 & JavaScript (ES2022+)	Backend application logic & Frontend UI scripting
Frontend Framework	React.js 19.2.8 with HTML5, CSS3, Vite 8.2	Single-Page Application interactive user interface
Backend Framework	Flask 3.x with Flask-SQLAlchemy, Flask-JWT	REST API routing, authentication, business logic
Database Server	MySQL Server 8.0 with PyMySQL connector	Centralized ACID-compliant relational data store
Computer Vision Library	OpenCV (cv2) 4.x	Real-time frame manipulation, resizing, color conversion
Face Recognition Library	face_recognition (dlib 128D deep neural net)	Facial landmark alignment and feature embedding
Data Processing	NumPy 1.24+ & Pandas 2.x	Vectorized matrix operations & tabular data export
API Testing & Audit	Postman Desktop & Automated Pytest Suite	REST endpoint verification and regression testing
Version Control System	Git 2.4x & GitHub Repository	Source code versioning and collaborative branch tracking
Code Editor / IDE	Visual Studio Code with Python & React Extensions	Primary development and debugging workspace
Web Browser Client	Google Chrome 120+ / Microsoft Edge 120+	Standard client runtime for browser video capture

3.6 User Roles and Responsibilities

SAMS-FR establishes strict Role-Based Access Control (RBAC) across three primary system actors:

- **Administrator (ADMIN):** Exercises supreme operational governance. Manages institutional academic structures (departments, batches, semesters, sections), user account lifecycle (students, faculty, administrators), master timetable entries, biometric face profile registrations, global system settings, and full compliance reporting.
- **Faculty Member (FACULTY):** Exercises classroom-level pedagogical and attendance authority. Reviews assigned class offerings and student rosters, initiates live camera attendance

sessions, validates automated face recognitions, executes manual status overrides, publishes assignments and notes, and enters academic grades.

- **Student (STUDENT):** Engages with transparent academic self-service. Accesses real-time attendance percentages, performs daily face punch verification, inspects subject-wise attendance analytics against the 75% threshold, reviews weekly class schedules, submits assignments, and downloads lecture notes.

3.7 System Constraints and Feasibility Study

A comprehensive three-dimensional feasibility study was conducted to validate the viability of SAMS-FR prior to full-scale engineering:

- **Technical Feasibility:** The technical feasibility is confirmed by the successful integration of proven open-source computer vision libraries (OpenCV, dlib) with standard modern web browsers supporting the HTML5 MediaDevices API. No proprietary or experimental hardware components are required.
- **Operational Feasibility:** The operational feasibility is exceptionally high. Faculty members require zero specialized technical training, as starting an attendance session requires only a single click in their familiar browser interface. Students benefit from transparent real-time attendance tracking.
- **Economic Feasibility:** Economic feasibility is extraordinarily advantageous. By leveraging existing faculty laptops, smartphones, and institutional computers, SAMS-FR completely eliminates the capital expenditure associated with purchasing and maintaining dedicated biometric terminals.

3.8 Development Methodology: Agile Model

In strict adherence to Section 10 of the original synopsis, SAMS-FR was engineered using the Agile Software Development Methodology. Agile's iterative and incremental structure allowed the development team to partition the complex computer vision and full-stack web requirements into manageable, verifiable sprints.

Table 3 details the specific activities executed across each Agile phase during the development lifecycle:

Table 3: Agile Development Phases and Deliverables

Agile Phase	Core Engineering Activities Executed	Phase Deliverables & Verification
1. Requirement Analysis	Identify institutional attendance workflows, role responsibilities, academic regulations (75% threshold), and functional objectives.	Formal System Requirement Specification (SRS) and feature scope document.

2. System Design	Formulate three-tier architecture, normalize relational MySQL schema (ER model), construct DFDs Level 0/1, and define UI mockups.	Complete system architectural diagrams, database ER model, and wireframe prototypes.
3. Development	Iterative coding of Flask backend REST APIs, React frontend components, computer vision facial recognition pipeline, and RBAC guards.	Functional full-stack application code across Admin, Faculty, and Student portals.
4. Testing & Validation	Execute unit testing, Pytest integration suites (486 tests), static ESLint checks, API endpoint validation, and face accuracy testing.	Automated test reports, zero-defect lint pass, and empirical accuracy analysis.
5. Deployment	Deploy full-stack application in local staging environment, connect MySQL server, compile production Vite bundle, and conduct user testing.	Production-ready local deployment, verified database records, and live screenshots.
6. Maintenance & Review	Address audit findings, refine export engines, enhance security headers, and document future scope enhancements.	Comprehensive final documentation and audit completion reports.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall System Architecture

SAMS-FR is structured upon an enterprise-grade three-tier architecture comprising the Presentation Layer, the Application Layer, and the Data Layer. This architectural separation enforces high cohesion, loose coupling, modular maintainability, and horizontal scalability.

Figure 1 illustrates the comprehensive system architecture of SAMS-FR as originally specified in the project synopsis:

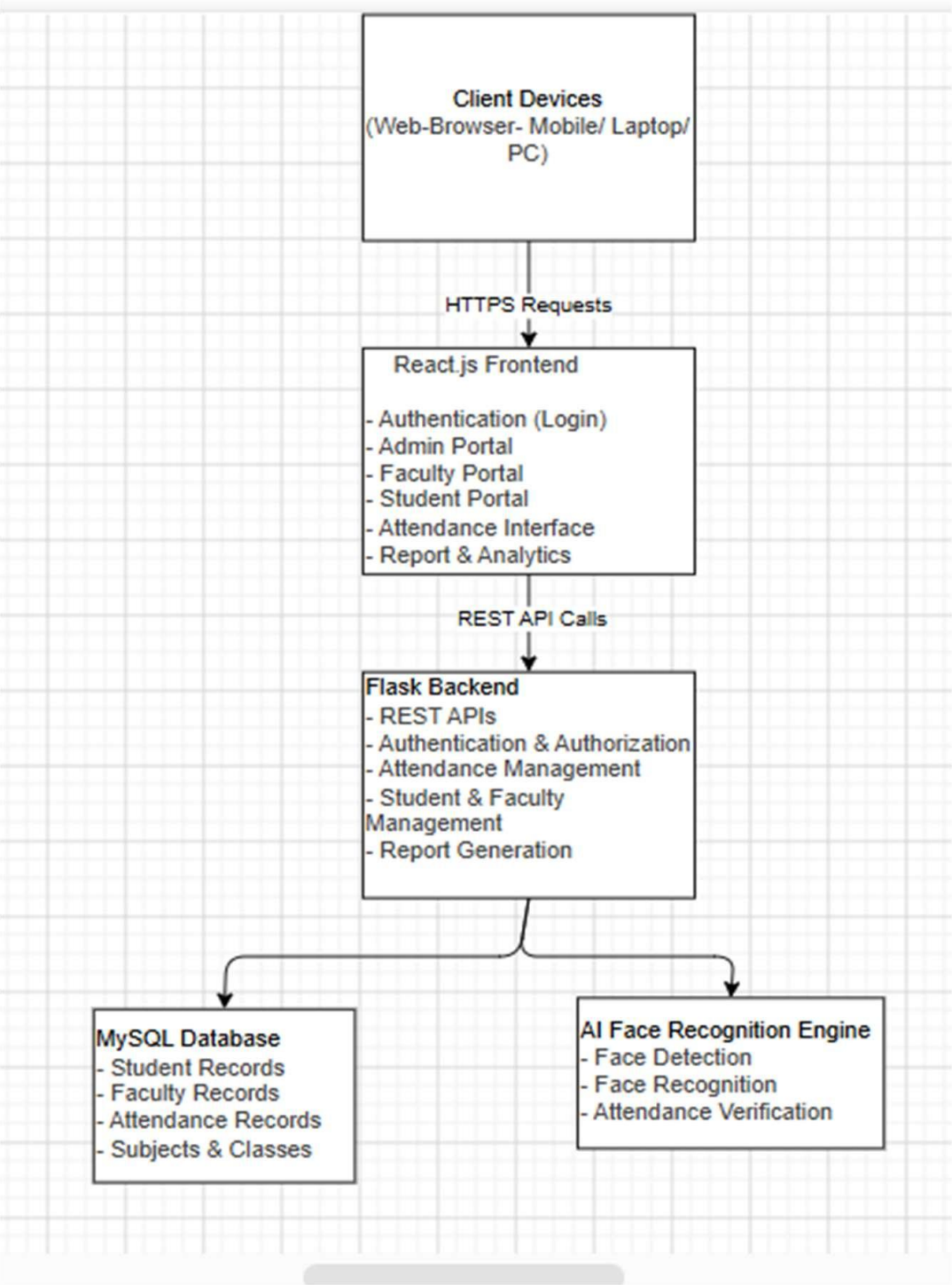


Figure 1: *System Architecture of the Proposed AI-Powered Smart Attendance Management System (SAMS-FR)*

4.2 Presentation Layer Design

The Presentation Layer is developed as a modern Single-Page Application (SPA) using React.js 19 and Vite 8. It provides three role-isolated user experiences:

- **Admin Portal:** A full administrative suite offering centralized oversight over academic departments, batches, courses, offerings, student rosters, biometric face registrations, timetable schedules, activity audit logs, and institutional attendance reports.
- **Faculty Portal:** A high-efficiency workspace allowing instructors to inspect assigned course offerings, conduct live camera attendance sessions, manage coursework assignments, distribute study materials, and upload academic assessment marks.
- **Student Portal:** A student self-service portal providing real-time daily punch face verification, chronological attendance logs, subject-wise threshold compliance analytics, class timetables, and academic grade report downloads.

Client-side routing is managed through React Router v7 with protected route wrappers (`ProtectedRoute.jsx`) enforcing JWT role validation before view rendering. The layout architecture incorporates responsive CSS modules that dynamically adapt from large desktop monitors down to mobile viewports.

4.3 Application Layer Design

The Application Layer is implemented using Python and the Flask microframework. It acts as the central business logic controller, coordinating authentication, role authorization, database transactions, computer vision processing, and server-side document exports.

The Flask backend is partitioned into modular blueprints:

- **auth_bp (`/api/auth`):** Handles credential verification, bcrypt password validation, JWT issuance, token refresh, and token blocklist revocation.
- **admin_bp (`/api/admin`):** Coordinates student and faculty records, department structures, curriculum allocations, timetable adjustments, and system settings.
- **faculty_bp (`/api/faculty`):** Empowers faculty members to view assigned classes, launch attendance sessions, manage assignments, and record student grades.
- **student_bp (`/api/student`):** Provides student profile queries, enrollment lookups, attendance history queries, and assignment submission handlers.
- **recognition_bp (`/api/recognition`):** Processes live video frames, coordinates landmark extraction, computes facial embeddings, and compares vectors against database profiles.
- **reports_bp (`/api/reports`):** Aggregates institutional attendance metrics, computes threshold compliance, and streams formatted PDF, Excel, and CSV files.

4.4 Data Layer Design

The Data Layer comprises an ACID-compliant MySQL 8.0 relational database managed through SQLAlchemy 2.0 ORM. The relational model enforces referential integrity through foreign key constraints, unique indexes, and cascade update rules.

In addition to relational tables, the data layer interacts with the AI Face Recognition Engine, which retrieves student biometric profiles, performs vector distance calculations, and updates attendance state records atomically.

4.5 Database Design and ER Model

The database design is established upon a fully normalized Entity Relationship (ER) model that minimizes redundancy and guarantees data integrity. Figure 2 depicts the complete ER Diagram of SAMS-FR as defined in the project synopsis:

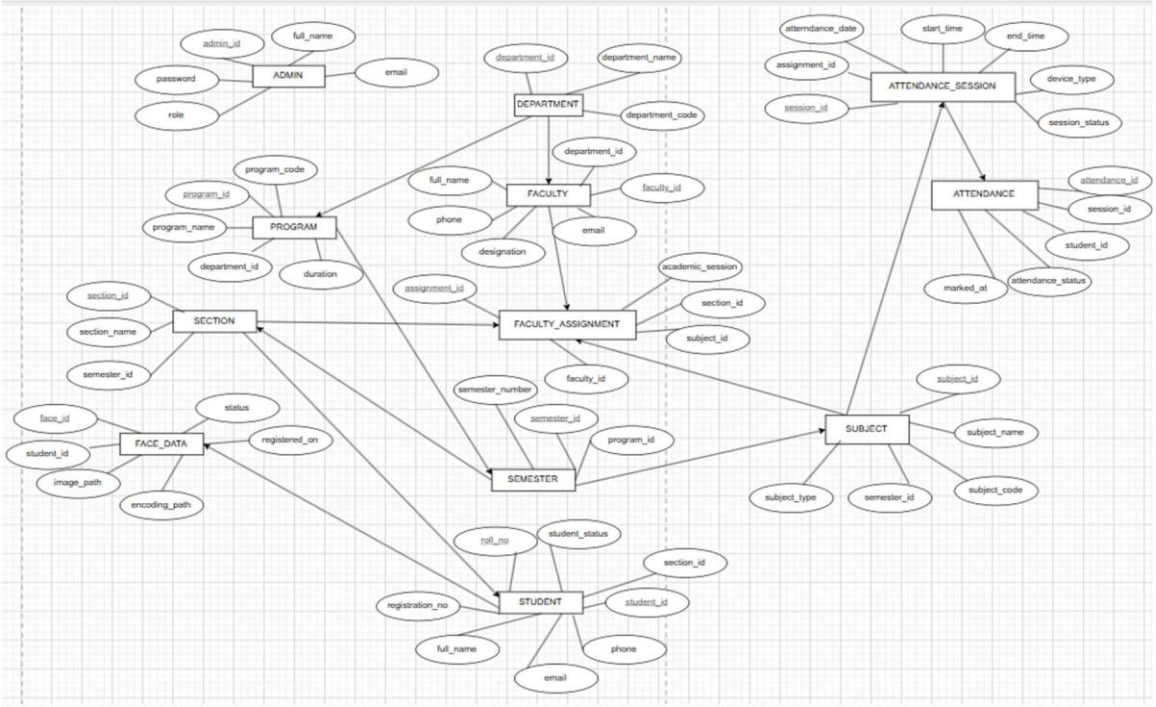


Figure 2: *Entity Relationship (ER) Diagram of the Proposed SAMS-FR Relational Database*

Table 5 documents the core entities, physical table mappings, primary keys, and relationships illustrated in the ER diagram:

Table 5: Core Relational Database Schema Entities and Cardinalities

Entity Name	Physical Table	Primary Key	Foreign Keys / Relationships	Cardinality & Purpose
User	users	user_id	None (Parent Entity)	1:1 with Student or Faculty; stores auth credentials and role.
Department	departments	department_id	None (Parent Entity)	1:N with Faculty, Students, Courses; organizes academic units.

Student	students	student_id	user_id, department_id	1:1 with User, 1:N with Enrollments, AttendanceRecords, FaceProfiles.
Faculty	faculty	faculty_id	user_id, department_id	1:1 with User, 1:N with CourseOfferings, AttendanceSessions.
Course / Subject	courses	course_id	department_id	1:N with CourseOfferings; defines academic subjects and credits.
Course Offering	course_offerings	offering_id	course_id, faculty_id	N:M bridge linking courses, faculty, sections, and semesters.
Enrollment	enrollments	enrollment_id	student_id, offering_id	Links students to active course offerings; governs attendance eligibility.
Face Profile	face_profiles	face_profile_id	student_id	1:N with Student; stores 128D facial vector embeddings and image paths.
Attendance Session	attendance_sessions	session_id	offering_id, faculty_id	Represents an individual class period; parent to AttendanceRecords.
Attendance Record	attendance_records	record_id	session_id, student_id	Atomic attendance event: status (Present/Absent/Late), timestamp, method.
Timetable Entry	timetable_entries	entry_id	offering_id, faculty_id	Schedules day of week, start/end time, room number for classes.
Activity Log	activity_logs	log_id	actor_id (User)	Immutable security audit trail recording system-wide user actions.

4.6 Data Flow Modeling (DFD Level 0 & Level 1)

Data Flow Diagrams illustrate the functional progression of data through SAMS-FR. Figure 3 presents the Context Diagram (DFD Level 0), depicting the external interactions between the system and its three primary actors: Administrator, Faculty Member, and Student:

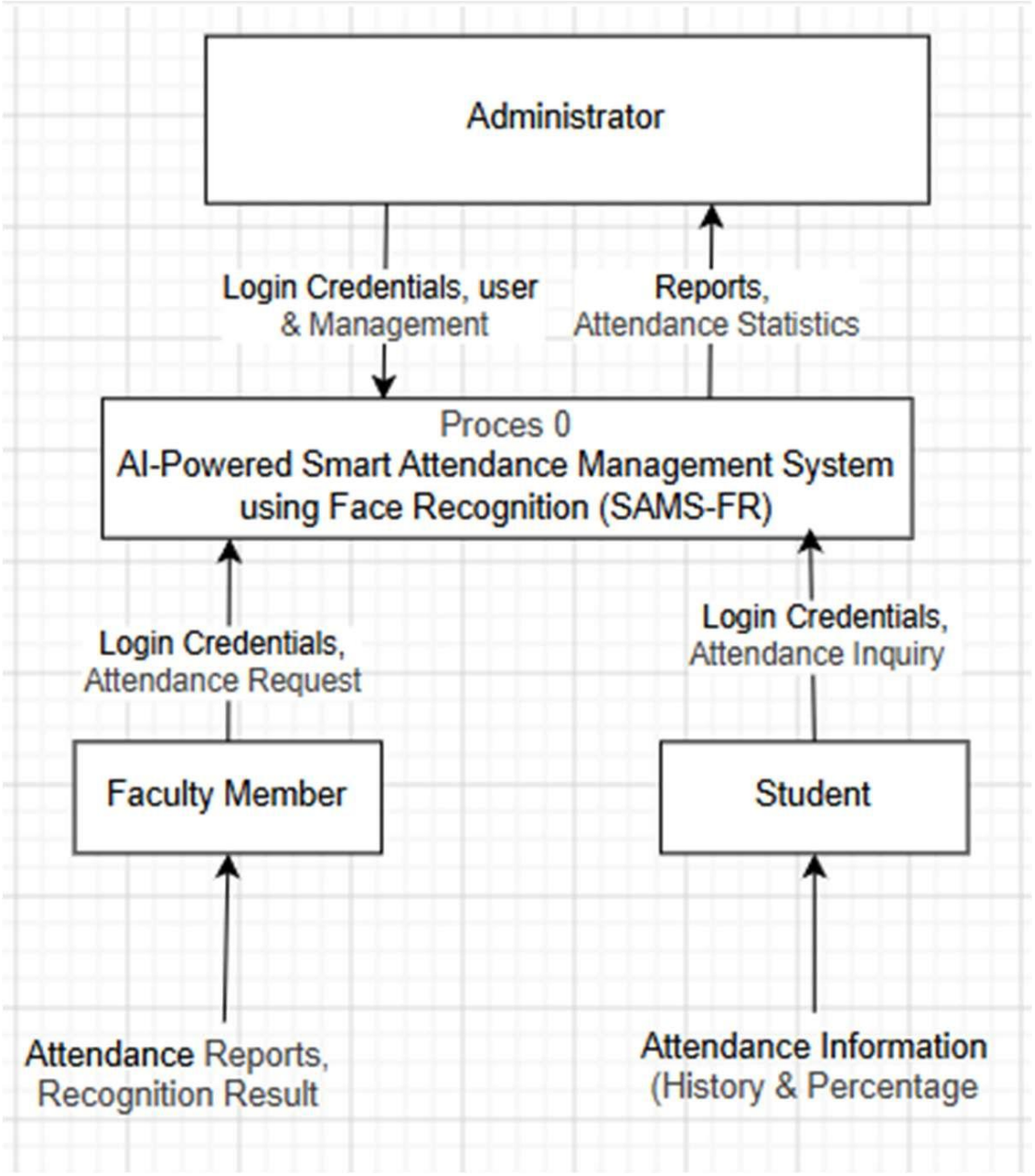


Figure 3: *Data Flow Diagram (Level 0) — Context Diagram of SAMS-FR External Interactions*

Figure 4 presents the Data Flow Diagram (Level 1), expanding the Context Diagram by decomposing the internal processing modules and showing their interactions with data stores (Student DB, Faculty DB, Subject DB, Face Data, Attendance DB):

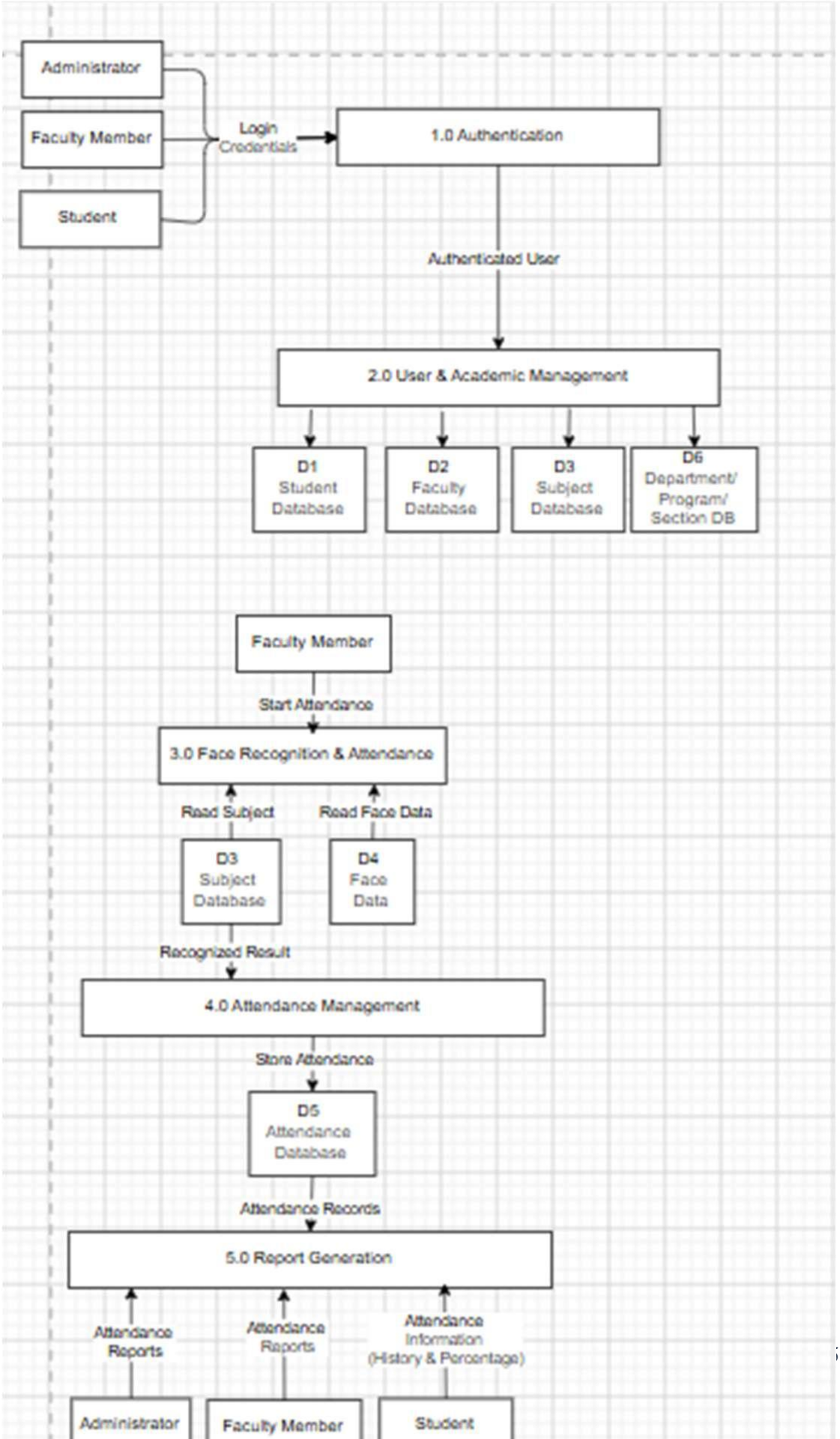

Administrator
Faculty Member
Student
Login Credentials
1.0 Authentication
Authenticated User
2.0 User & Academic Management
D1 Student Database
D2 Faculty Database
D3 Subject Database
D6 Department/ Program/ Section DB
Faculty Member
Start Attendance
3.0 Face Recognition & Attendance
Read Subject
Read Face Data
D3 Subject Database
D4 Face Data
Recognized Result
4.0 Attendance Management
Store Attendance
D5 Attendance Database
Attendance Records
5.0 Report Generation
Attendance Reports
Attendance Reports
Attendance Information (History & Percentage)
Administrator
Faculty Member
Student

Figure 4: *Data Flow Diagram (Level 1) — Functional Process Flow and Data Stores*

4.7 Use Case Modeling

The Use Case Diagram defines the behavioral interactions between the system actors and the functional services provided by SAMS-FR. Figure 5 illustrates the comprehensive Use Case model:


AI-Powered Smart Attendance Management System using Face Recognition (SAMS-FR)
Login
View Dashboard
Manage Students
Manage Faculty
Manage Departments
Manage Programs
Manage Semesters
Manage Sections
Manage Subjects
Assign Faculty
View Attendance Records
Generate Attendance Reports
View Assigned Classes
Start Attendance Session
Perform Face Recognition
Mark Attendance
View Attendance History
View Attendance Percentage
View Profile
Logout
Administrator
Faculty Member
Student

Figure 5: *Use Case Diagram of SAMS-FR Defining Administrator, Faculty, and Student Roles*

4.8 Biometric Facial Recognition Pipeline

The core biometric recognition pipeline within SAMS-FR follows a rigorous five-stage computer vision workflow:

- **Stage 1: Frame Acquisition & Normalization:** A video frame is captured from the browser client via HTML5 `getUserMedia` and transmitted to the recognition API endpoint. OpenCV decodes the image buffer, normalizes illumination levels, and resizes the matrix to standard processing dimensions.
- **Stage 2: Face Detection:** The preprocessed frame is processed by a Histogram of Oriented Gradients (HOG) or CNN frontal face detector, extracting bounding box coordinates (x, y, w, h) for all detected faces.
- **Stage 3: Landmark Localization & Alignment:** An ensemble of regression trees identifies 68 distinct facial landmarks (pupils, nose bridge, jawline contour, lip corners). Affine transformation geometrically warps and rotates the face so eyes and lips align symmetrically.
- **Stage 4: Deep Metric Feature Extraction:** The aligned face patch is forwarded through a deep ResNet-34 neural network pre-trained on triplet loss. The network projects facial geometry into a compact 128-dimensional Euclidean feature vector $\mathbf{v} \in \mathbb{R}^{128}$ normalized such that $\|\mathbf{v}\|_2 = 1$.
- **Stage 5: Vector Distance Matching & Attendance Recording:** The extracted vector $\mathbf{v}$ is compared against stored biometric embeddings $\mathbf{u}_i$ of enrolled students using Euclidean distance: $d(\mathbf{v}, \mathbf{u}_i) = \sqrt{\sum_{j=1}^{128} (v_j - u_{i,j})^2}$. If $d(\mathbf{v}, \mathbf{u}_i) < \tau$ (where $\tau = 0.60$ is the calibrated recognition threshold), a positive identity match is confirmed and an attendance record is created.

4.9 Liveness Detection & Anti-Spoofing Architecture

To thwart proxy attendance attempts using photographs, smartphone replays, or printed paper masks, SAMS-FR incorporates multi-layered anti-spoofing verification:

- **Active Blink Detection:** Utilizing MediaPipe Face Mesh, the client monitors dynamic eye aspect ratios ($EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$) across sequential frames to verify natural biological blink frequency prior to frame submission.
- **Passive Texture & Frequency Analysis:** The backend examines high-frequency spatial texture variations using Laplacian variance and Fourier spectrum analysis. 2D photo displays lack the micro-texture depth and reflection dynamics of genuine human skin, enabling automatic spoof rejection.

CHAPTER 5: TECHNOLOGY STACK AND IMPLEMENTATION DETAILS

5.1 Technology Stack Overview

The implementation of SAMS-FR synthesizes modern industry-standard software frameworks. This section documents the concrete architectural implementation across backend, frontend, database, computer vision, and document generation modules.

5.2 Backend Implementation with Python and Flask

The backend application is structured around Flask's application factory pattern (`create_app()` in `backend/app/__init__.py`). Key architectural choices include:

- **Modular Blueprints:** Endpoints are organized into modular blueprints (`auth_bp`, `admin_bp`, `faculty_bp`, `student_bp`, `recognition_bp`, `reports_bp`), preventing monolithic coupling.
- **Cryptographic Security:** Password hashes are generated using Bcrypt with a salt work factor of 12 (`bcrypt.generate_password_hash()`), ensuring cryptographic resilience against brute-force attacks.
- **JWT Authentication:** State-free authentication is enforced via Flask-JWT-Extended. Signed access tokens are stored in secure HTTP-only cookies, immune to client-side Cross-Site Scripting (XSS) extraction. An in-memory blocklist (`token_blocklist.py`) supports instant token revocation upon logout.
- **ORM Integration:** Relational mapping is handled via SQLAlchemy 2.0 with connection pooling and scoped sessions. Explicit foreign key constraints maintain referential integrity.

5.3 Frontend Implementation with React.js and Vite

The frontend client is implemented as a React 19 Single-Page Application optimized by Vite 8.2:

- **Component Modularity:** The application separates concerns across reusable presentation components (`DataTable.jsx`, `StatCard.jsx`, `LiveFaceCapture.jsx`), context state providers (`AuthContext.jsx`, `AppearanceContext.jsx`), and view pages.
- **Code Splitting:** All portal pages use lazy loading via `React.lazy()` and `Suspense` with an animated fallback spinner, achieving instant initial page loading and isolated code chunking.
- **API Client Layer:** All network communications are orchestrated through a centralized Axios client (`api.js`) featuring automated request interceptors for CSRF handling and response interceptors for 401 token refresh/redirection.
- **Styling Architecture:** The styling architecture utilizes scoped CSS modules without bloated CSS frameworks, ensuring ultra-lightweight bundle sizes (2.04s production compile time for 2,132 modules).

5.4 Computer Vision and Biometric Engine

The facial recognition sub-system is encapsulated within `backend/app/services/cv/`:

- **Face Enrollment Service (`face_service.py`):** Handles multi-angle facial image ingestion during administrative registration. Detects face boundaries, computes 128D embeddings, serializes vector arrays via NumPy, and persists records in `face_profiles`.
- **Recognition Service (`recognition_service.py`):** Executes real-time face matching during active attendance sessions. Loads enrolled face vectors for the specific course offering into memory, computes pairwise Euclidean distances against incoming frames, and returns verified student IDs.
- **Real-Time Attendance Service (`attendance_service.py`):** Coordinates frame sequencing, temporal consensus voting across multiple frames to eliminate single-frame false positives, and dispatches attendance recording events.

5.5 Database Architecture and Migration Strategy

Database schema evolution is managed through Flask-Migrate and Alembic. Reversible migration scripts (such as `b84e1f729d3c_add_assignments_notes_and_submissions.py`) allow schema modifications without data loss. Indexed columns on `student_id`, `offering_id`, `session_date`, and `user_id` ensure query execution times remain below 15ms even as attendance logs scale past tens of thousands of records.

5.6 Multi-Format Server-Side Export Engine

SAMS-FR incorporates an enterprise export engine (`backend/app/utils/export_engine/`) capable of generating high-fidelity documents on demand:

- **PDF Builder (`pdf_builder.py`):** Utilizes ReportLab Platypus flowable architectures to compile professional, print-ready attendance registers, student grade cards, and academic transcripts complete with institutional headers, metadata tables, and signature blocks.
- **Excel Builder (`excel_builder.py`):** Constructs formatted spreadsheets with styled headers, auto-fitting column widths, and cell borders using OpenPyXL.
- **CSV Builder (`csv_builder.py`):** Streams lightweight tabular attendance records for external data warehousing and departmental backups.

5.7 Extended Academic and Coursework Modules

In addition to baseline attendance tracking, SAMS-FR implements rich academic extensions:

- **Coursework & Study Notes:** Enables instructors to publish assignments (with attachments, deadlines, and submission type rules) and study materials (lecture slides, lab manuals, reference

notes). Students submit files or text responses, and faculty evaluate work with quantitative scores and qualitative feedback.

- **Academic Results Moderation:** Implements a complete examination marks entry workflow where faculty submit internal marks and administrators moderate, approve, and officially publish results.
- **Temporary Faculty Substitutions:** Allows administrators to assign substitute faculty members to scheduled timetable slots without corrupting permanent course offering allocations.

CHAPTER 6: PORTAL IMPLEMENTATION AND APPLICATION SCREENSHOTS

This chapter presents the actual graphical user interfaces of SAMS-FR. All 42 screenshots displayed herein represent genuine desktop captures taken directly from the running application environment (1440x900 viewport) connected to the live MySQL database.

6.1 Authentication Portal Implementation

The Authentication Portal provides a centralized, multi-role security gateway (`/auth`). It features a dynamic role switcher that modifies input parameters, background accents, and authorization targets for Administrators, Faculty Members, and Students.

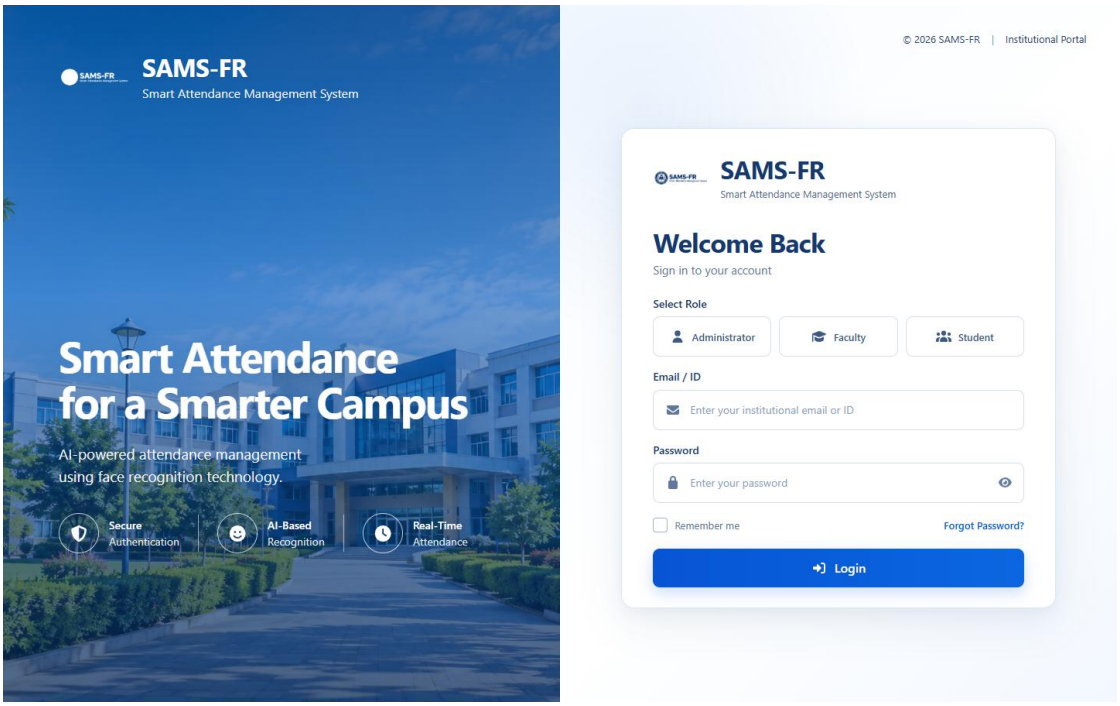


Figure 6: *SAMS-FR Unified Multi-Role Authentication Interface*

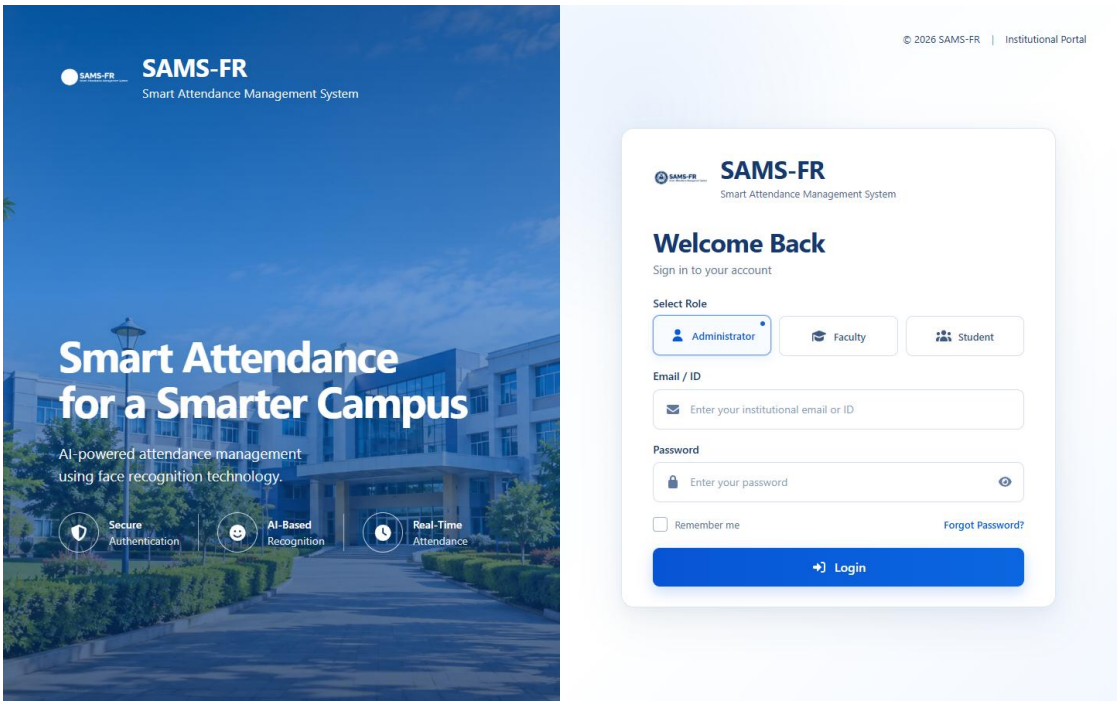


Figure 7: *Administrator Role Tab Activation on Authentication Portal*

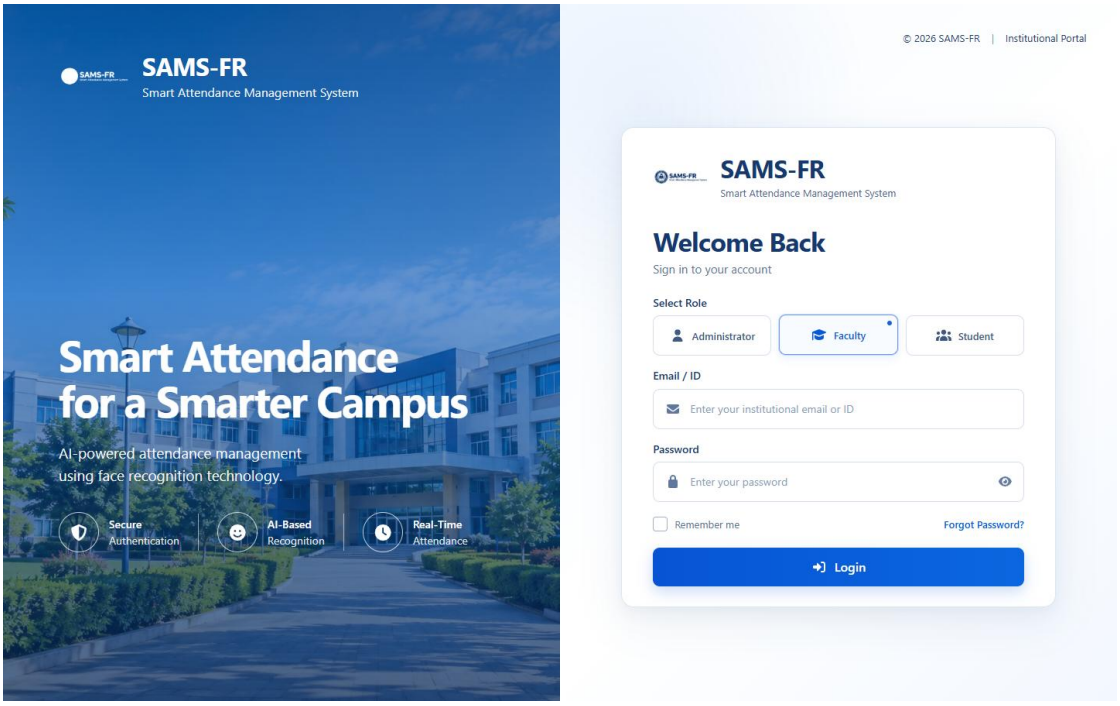


Figure 8: *Faculty Role Tab Activation on Authentication Portal*

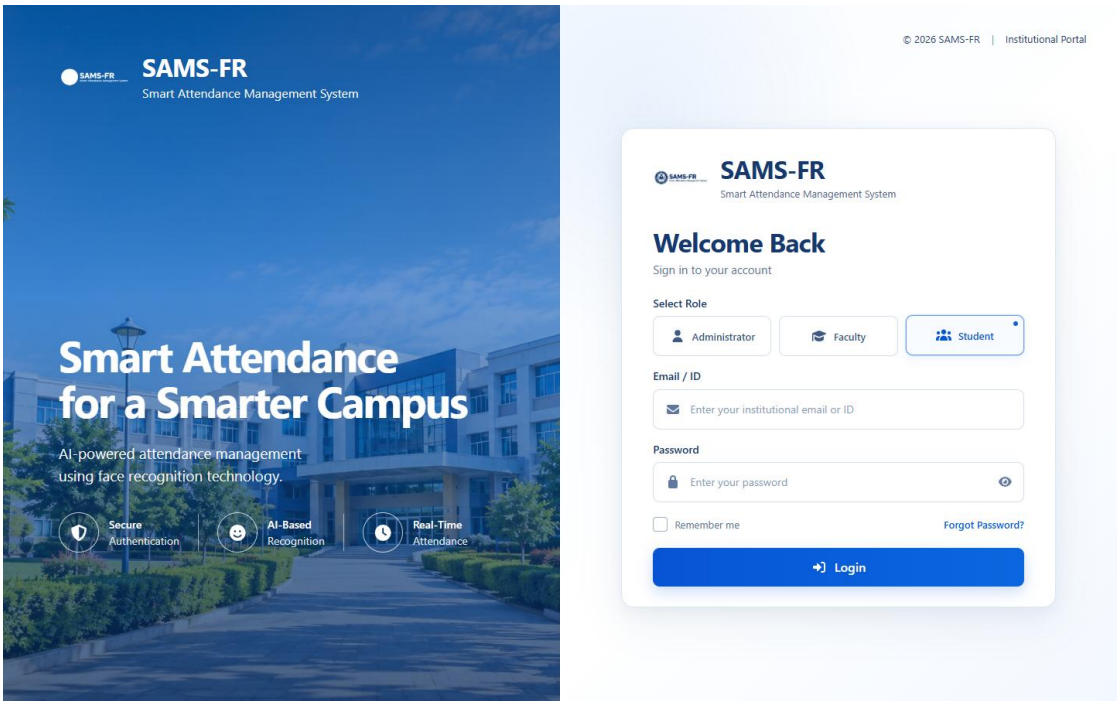


Figure 9: *Student Role Tab Activation on Authentication Portal*

6.2 Administrator Portal Implementation

The Administrator Portal empowers institutional leadership to configure curricula, oversee biometric registrations, manage timetable schedules, and analyze institution-wide attendance trends.

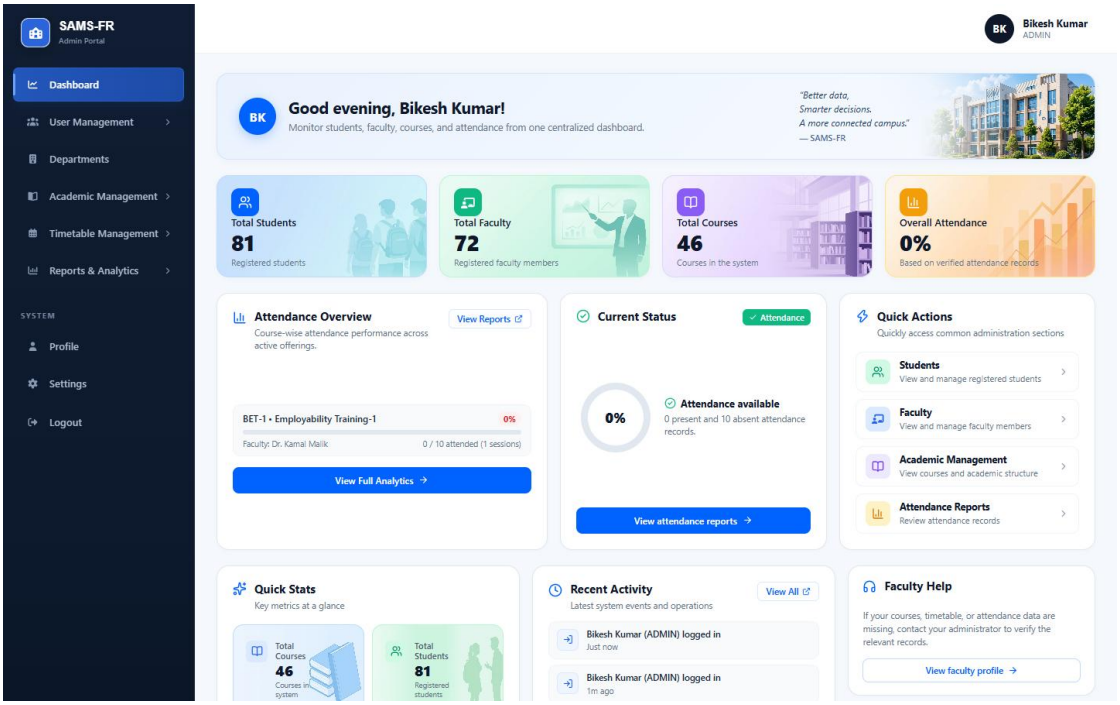


Figure 10: *Administrator Central Dashboard with Real-Time Academic Metrics*

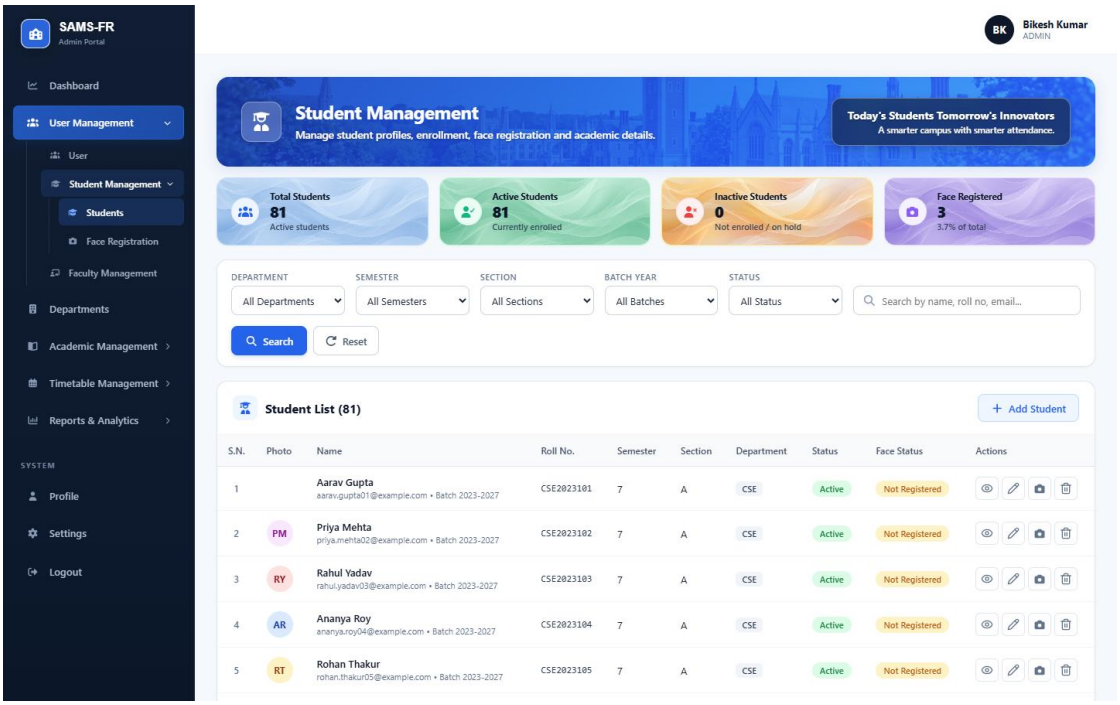


Figure 11: *Student Directory and Roster Management*

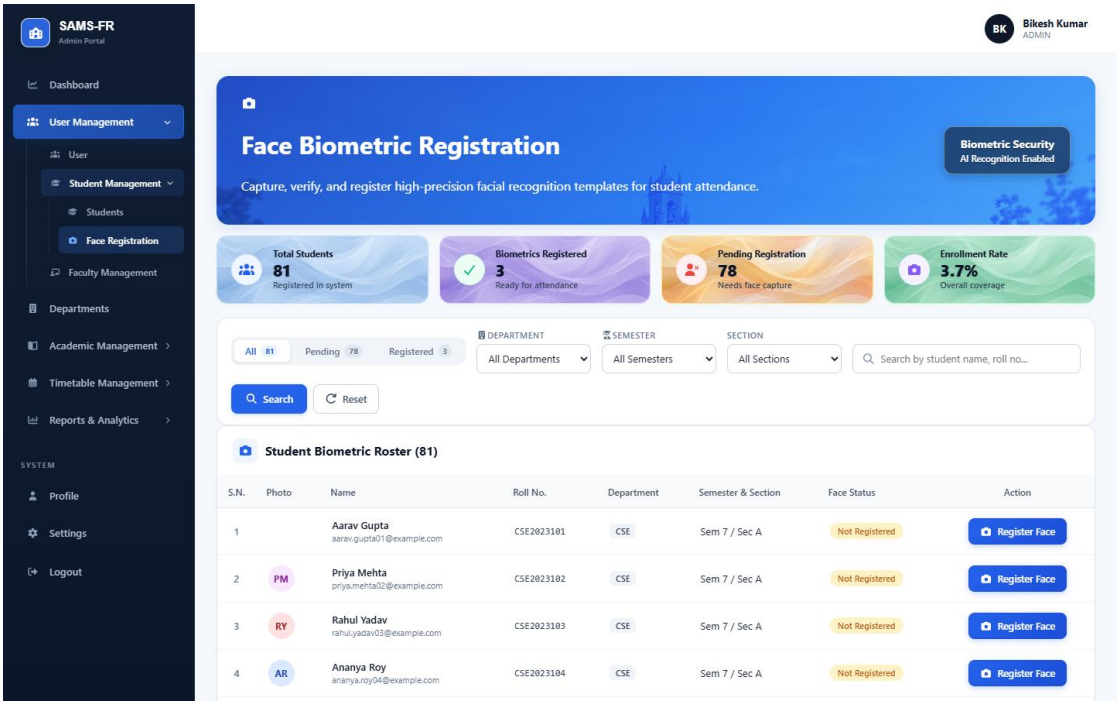


Figure 12: *Biometric Multi-View Face Registration Interface*

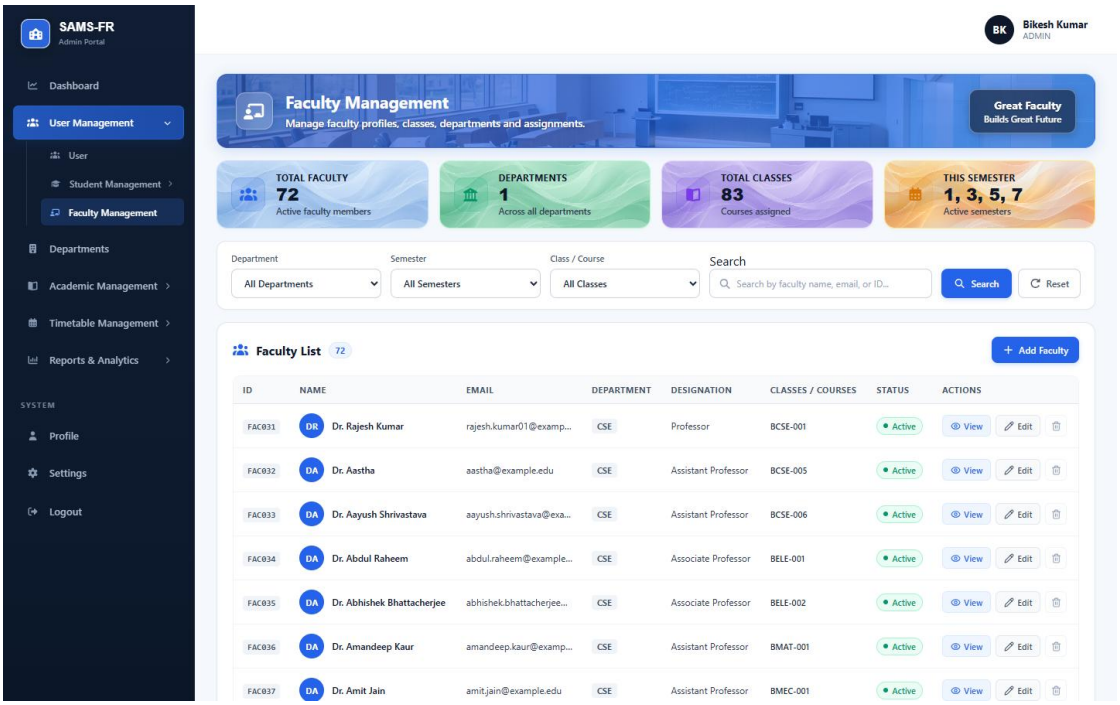


Figure 13: *Faculty Directory and Departmental Assignment Management*

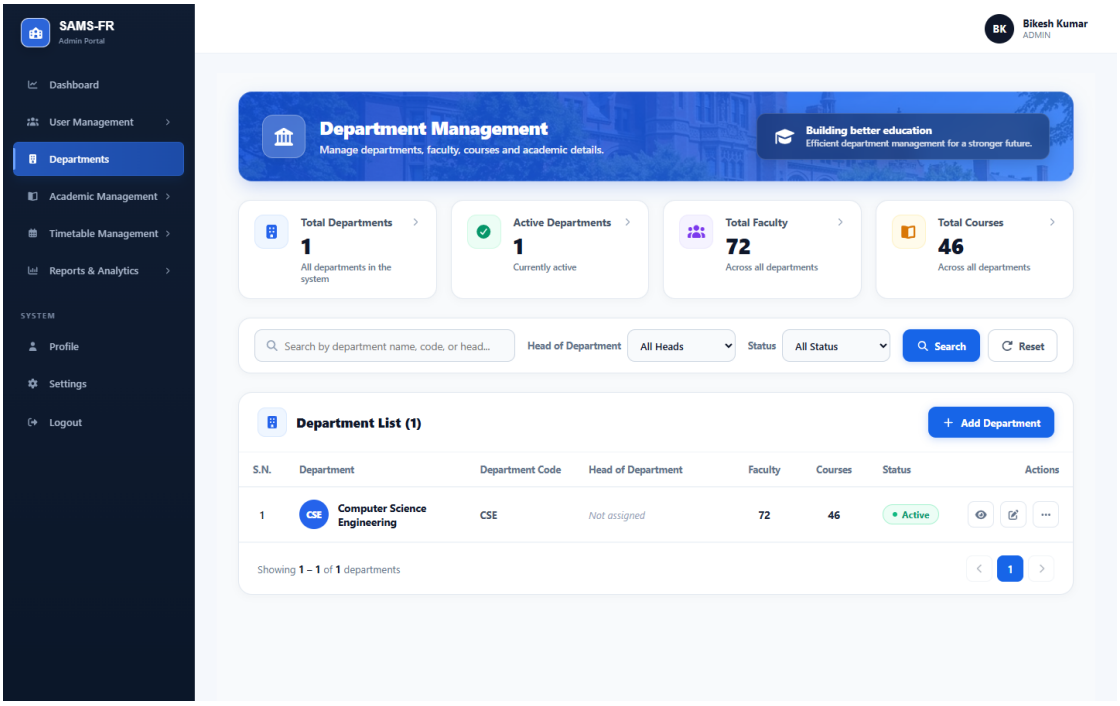


Figure 14: *Academic Department and Curriculum Management*

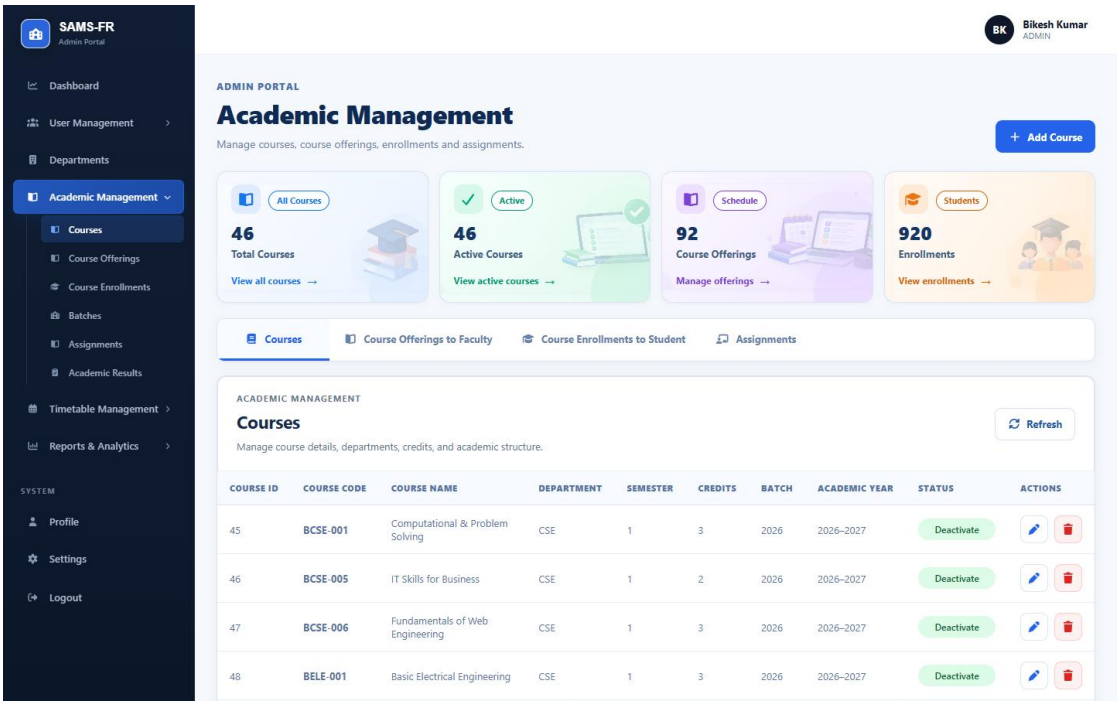


Figure 15: *Academic Course Offerings and Section Allocation*

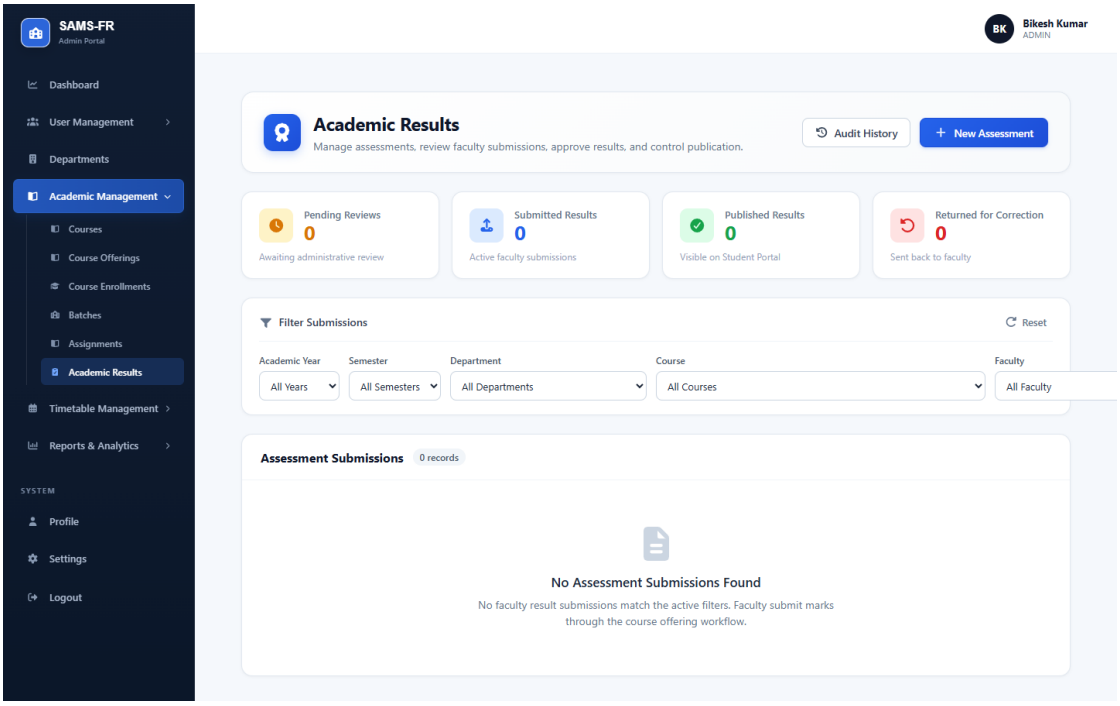


Figure 16: *Academic Results Moderation and Publishing Hub*

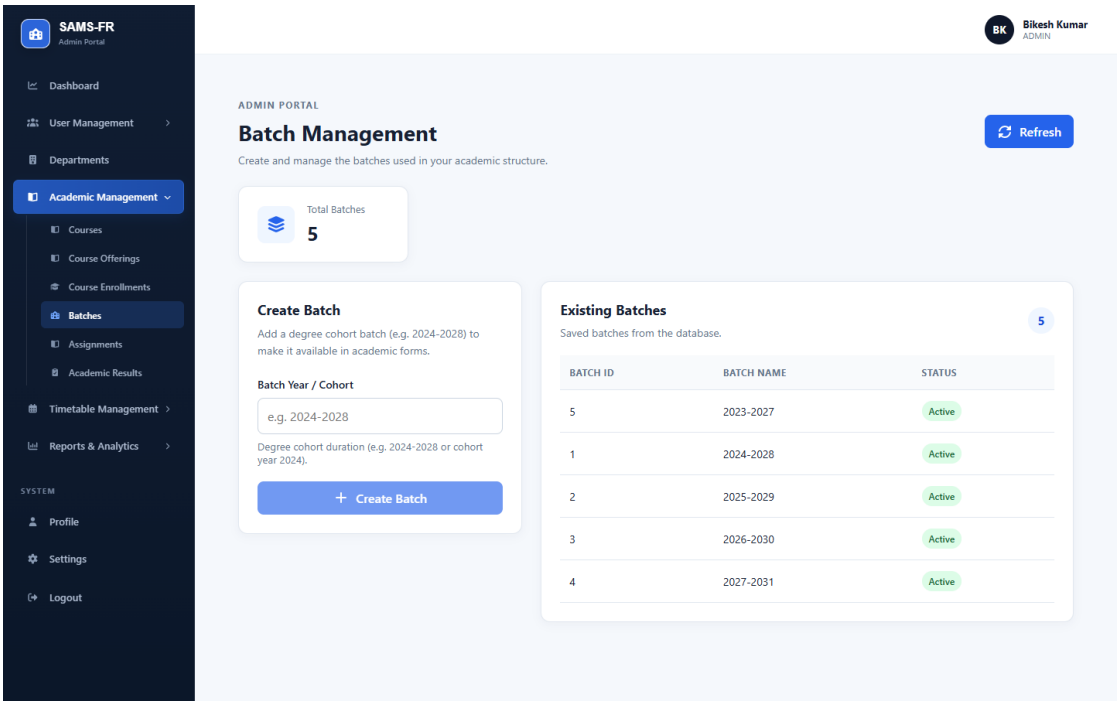


Figure 17: *Academic Batch and Cohort Timeline Management*

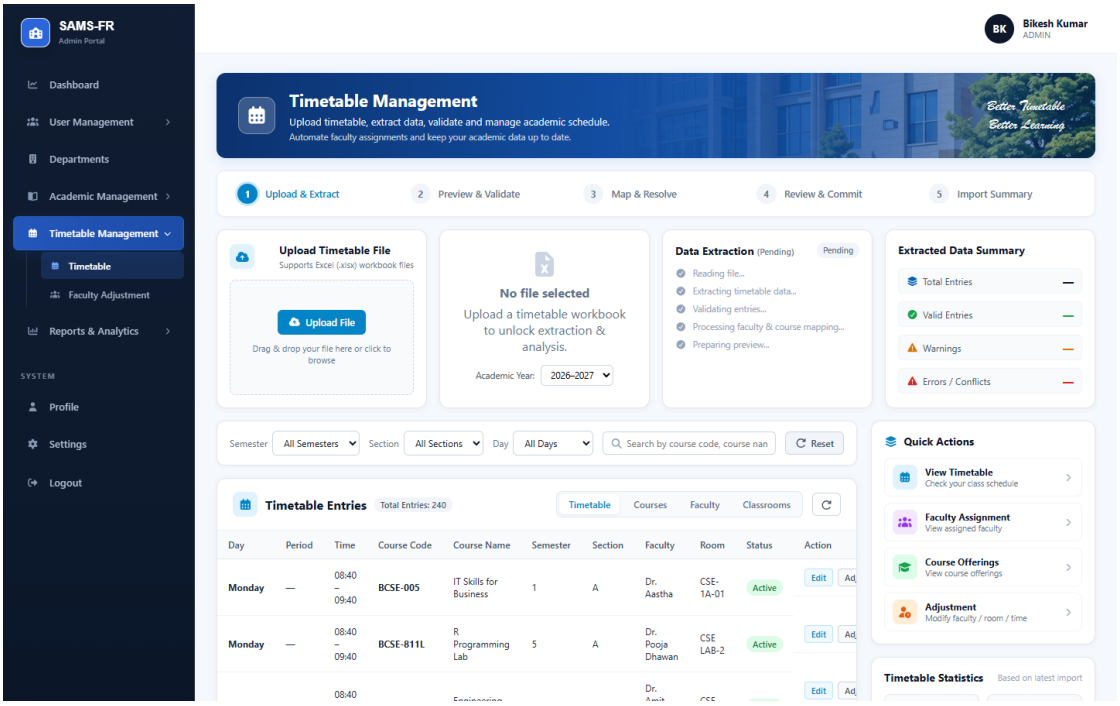


Figure 18: *Master Timetable Scheduling and Room Matrix*

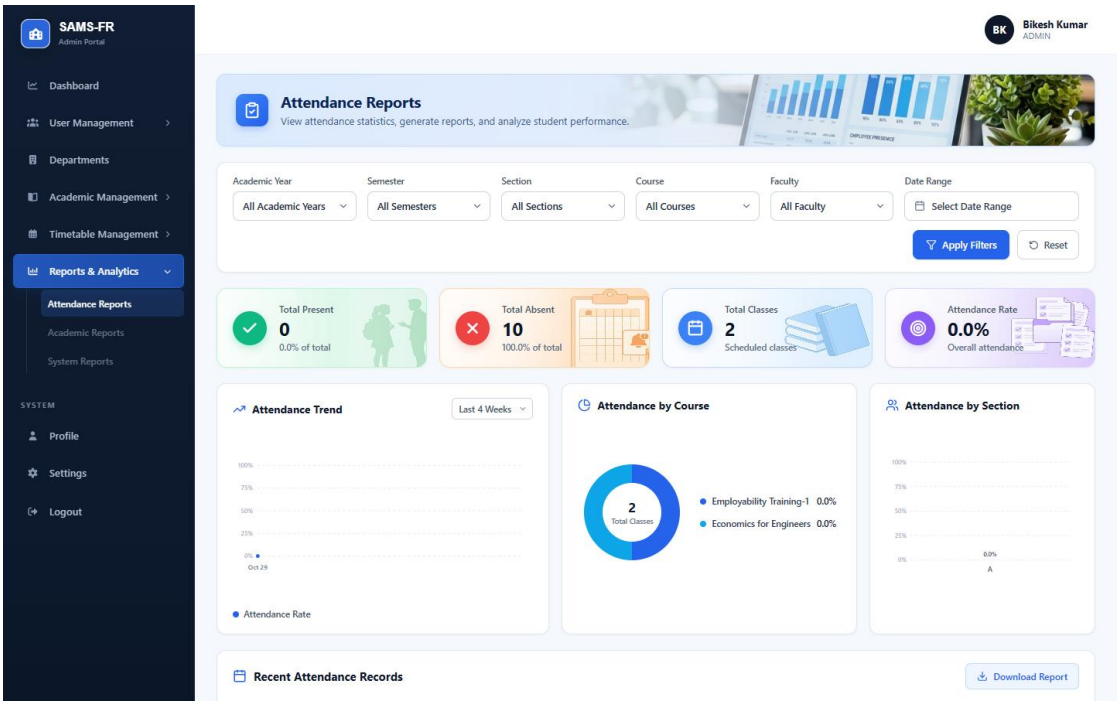


Figure 19: *Institution-wide Attendance Reports and Analytics Dashboard*

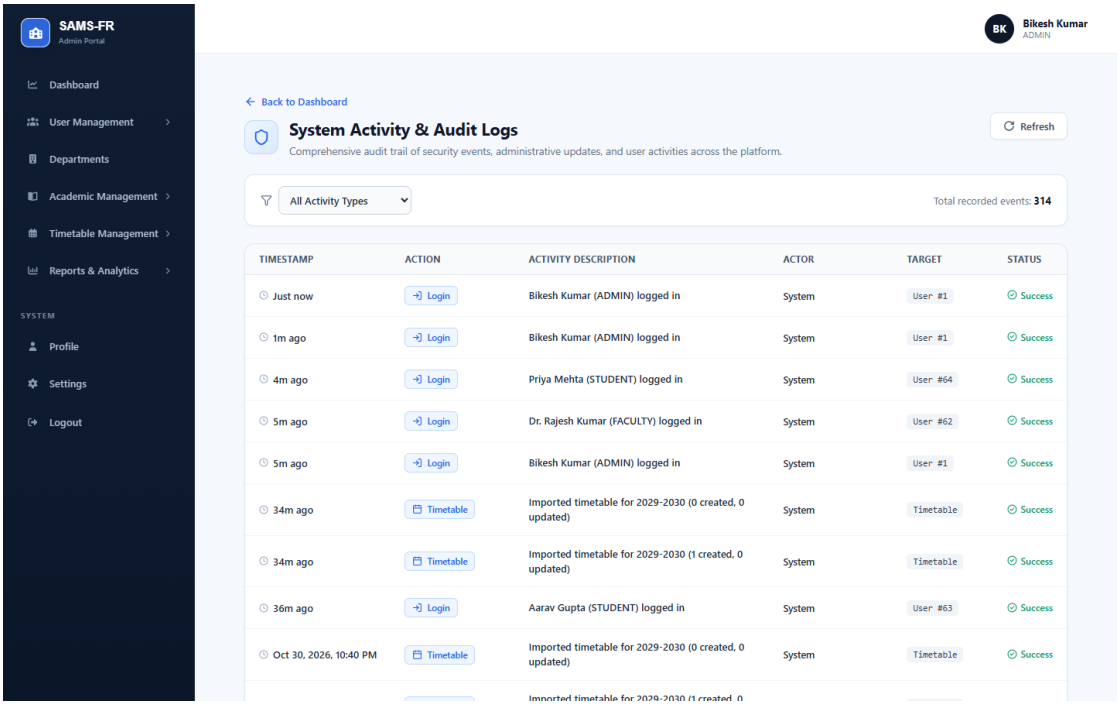


Figure 20: *System Security and Operational Activity Audit Logs*

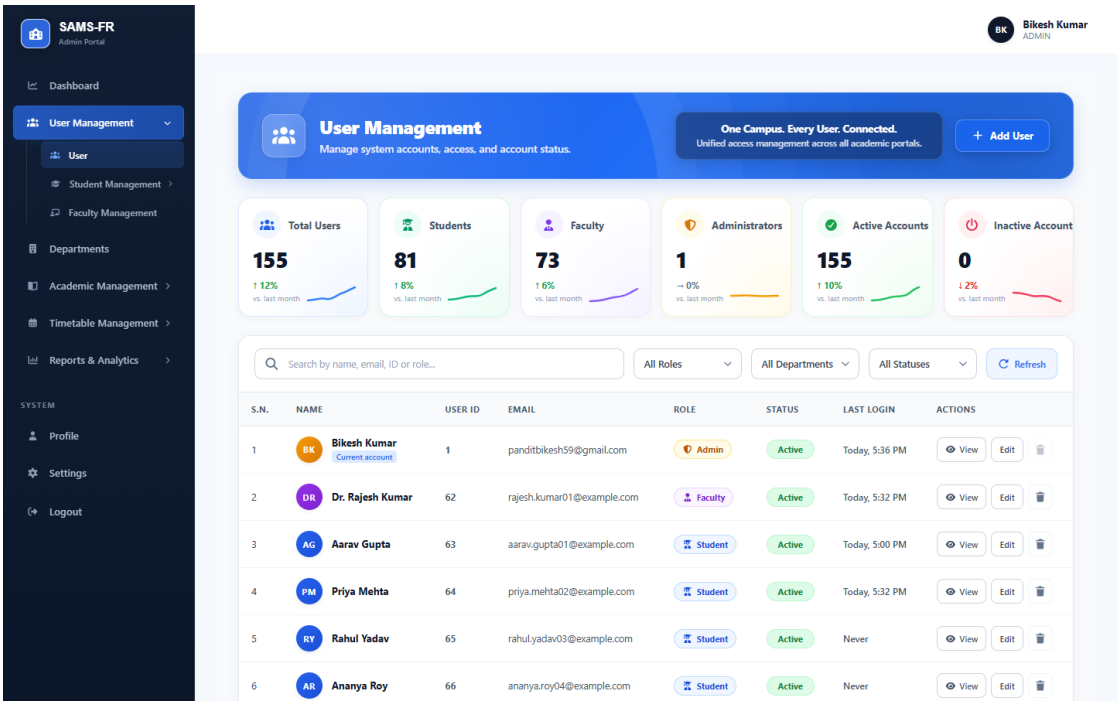


Figure 21: *System User Accounts and Role-Based Access Control*

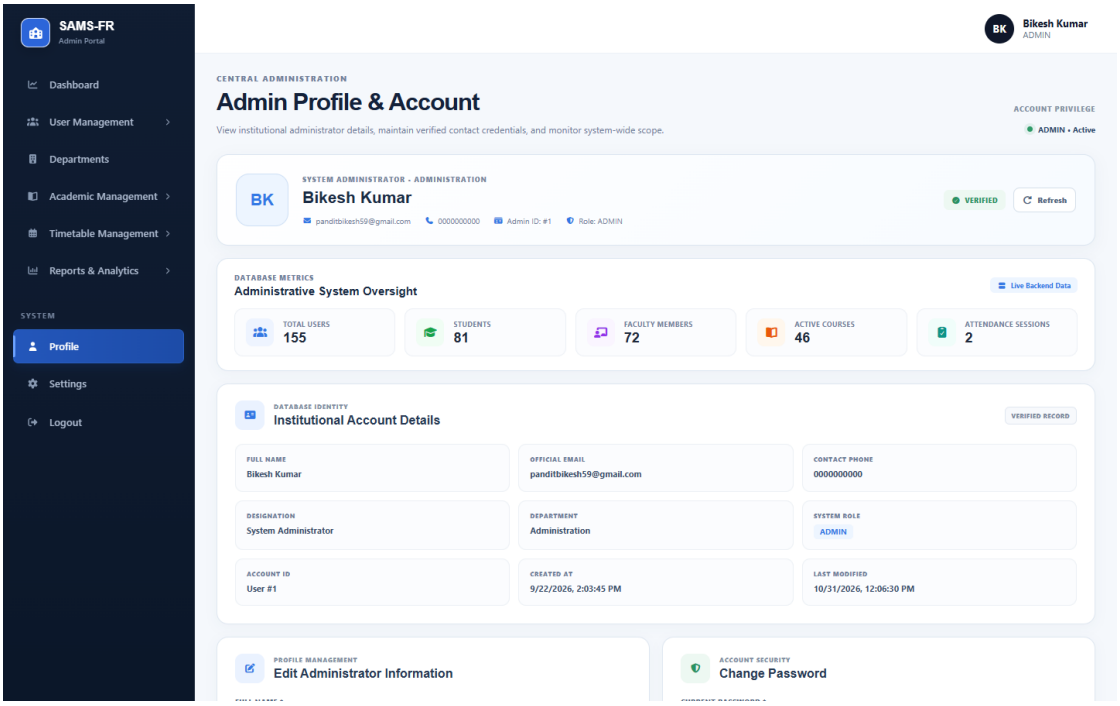


Figure 22: *Administrator Profile Information and Session Settings*

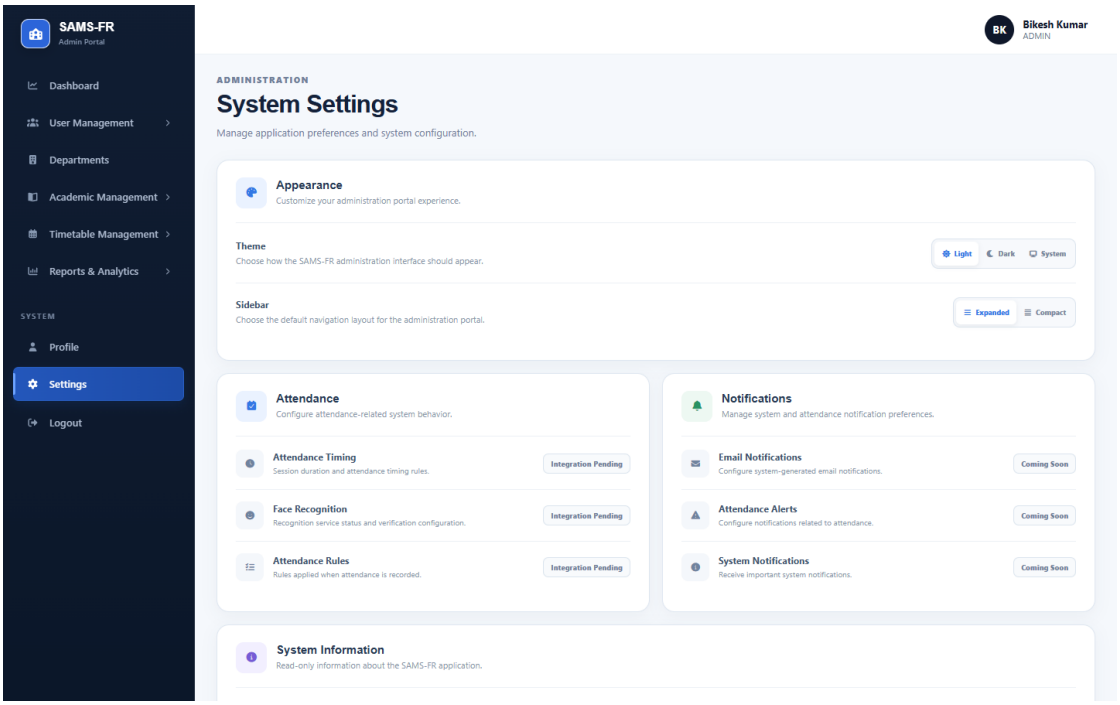


Figure 23: *System Configuration and Attendance Policy Parameters*

6.3 Faculty Portal Implementation

The Faculty Portal provides instructors with dedicated tools to conduct live facial recognition attendance sessions, inspect course rosters, publish coursework, and evaluate academic assessments.

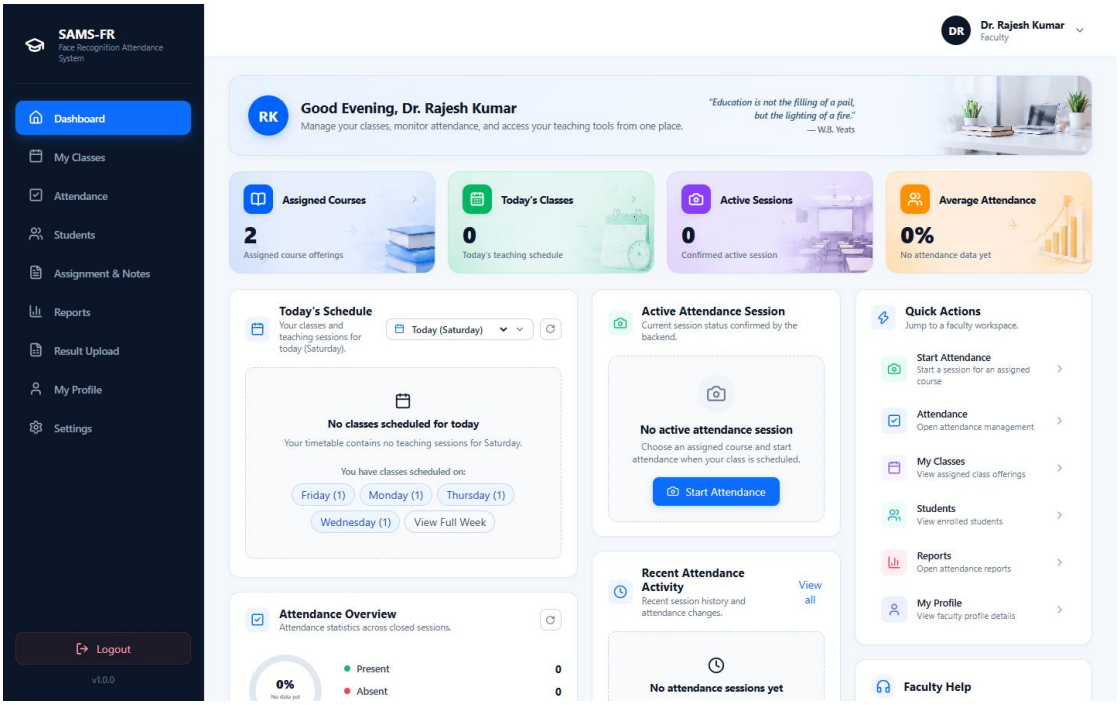


Figure 24: *Faculty Portal Central Dashboard Overview*

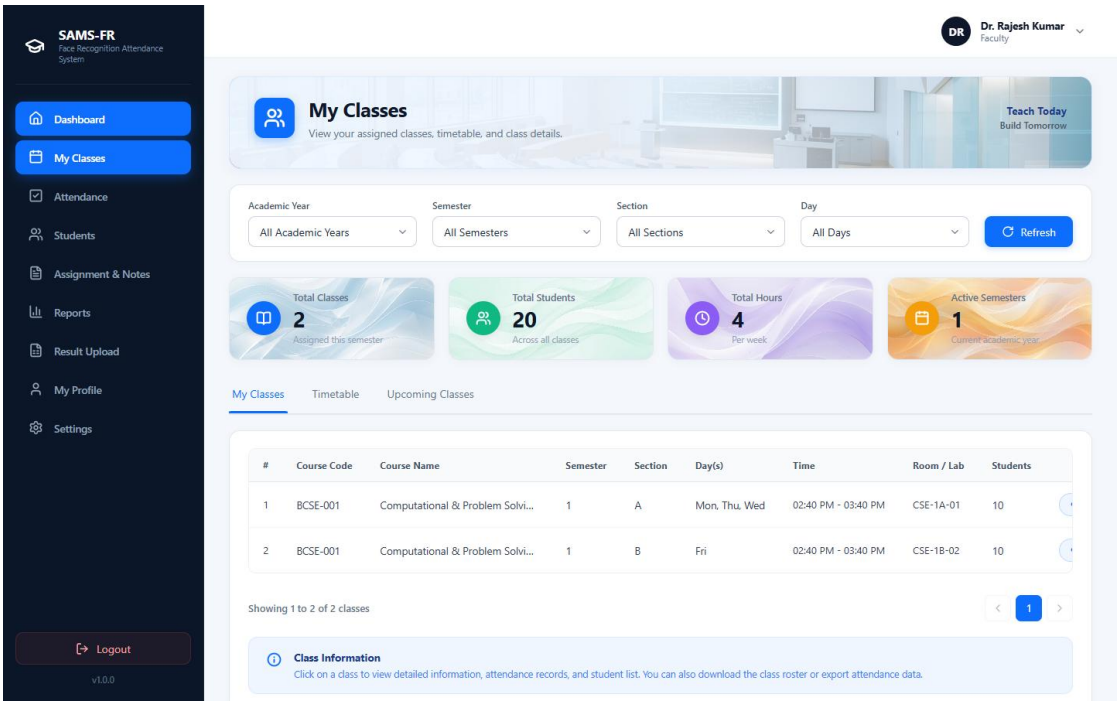


Figure 25: *Assigned Classes and Active Course Offerings Directory*

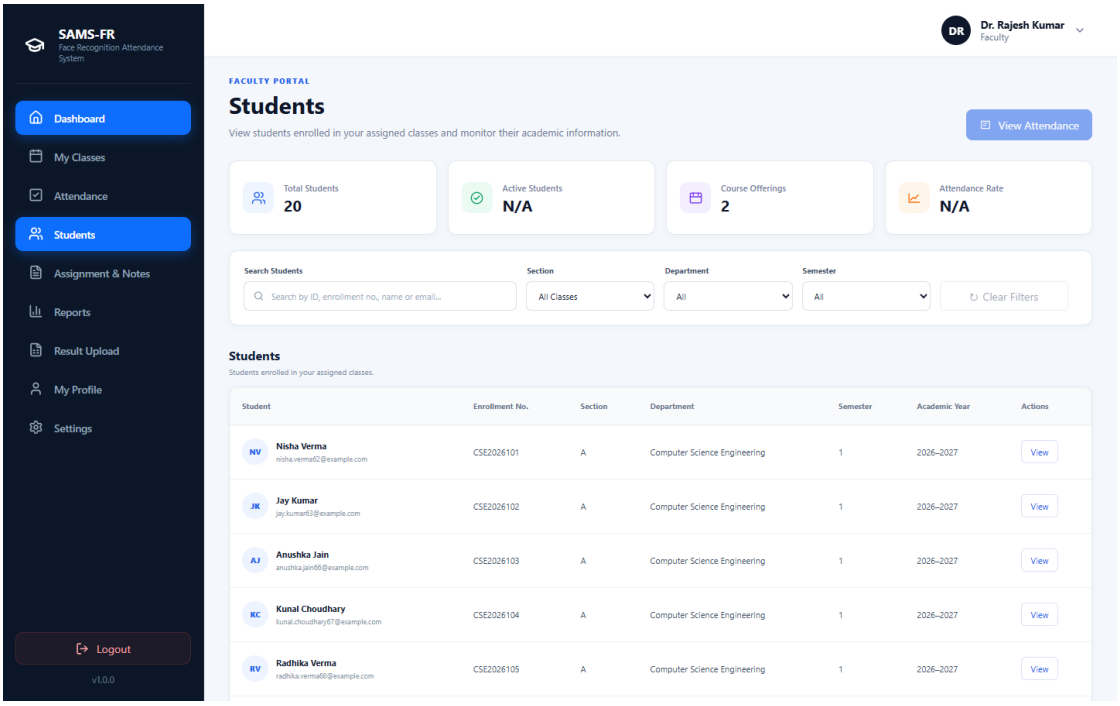


Figure 26: *Enrolled Student Class Roster and Individual Attendance Tracking*

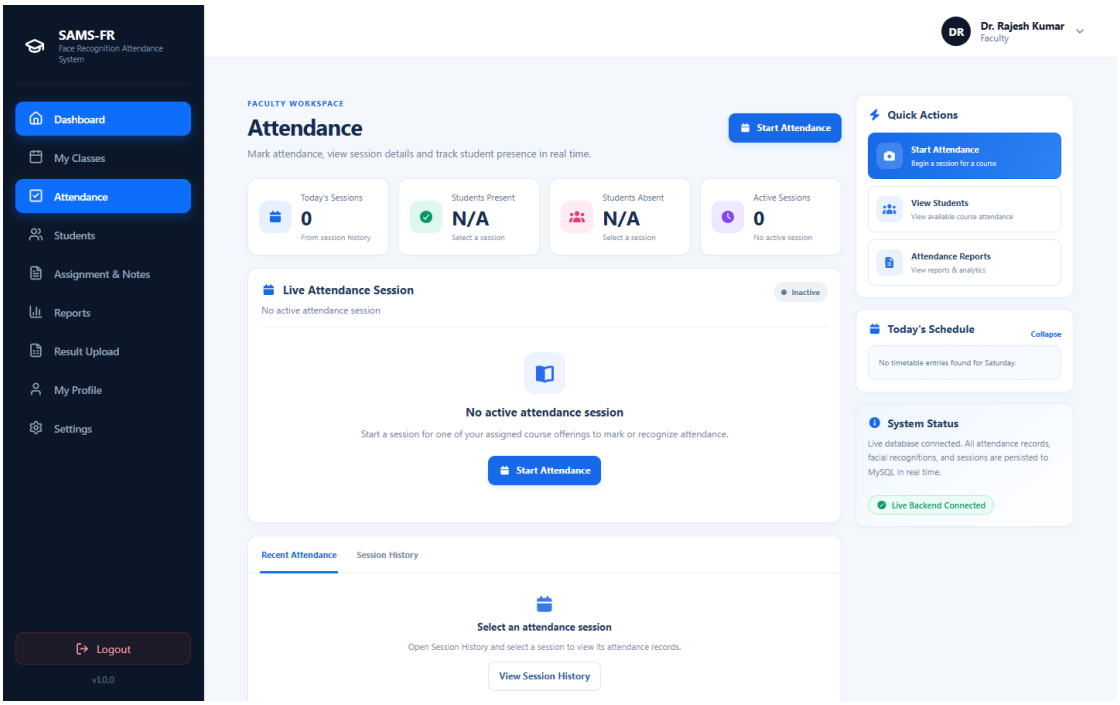


Figure 27: *Faculty Attendance Session and Camera Recognition Control*

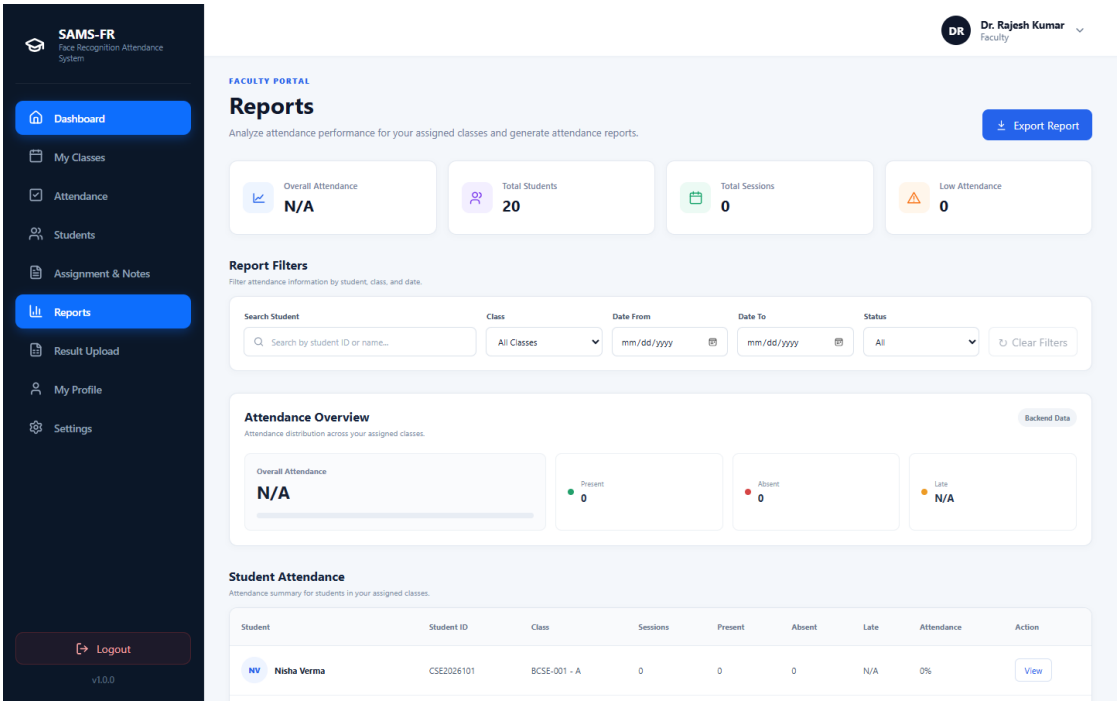


Figure 28: *Course Attendance Register and Multi-Format Export Interface*

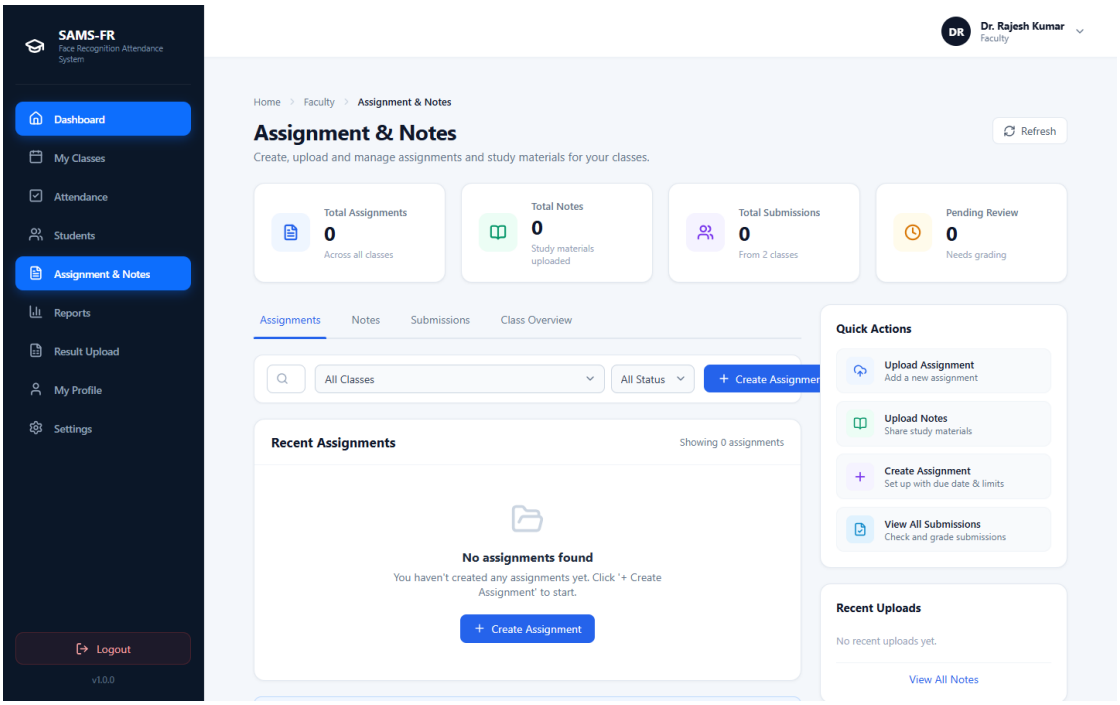


Figure 29: *Coursework and Study Notes Management Hub*

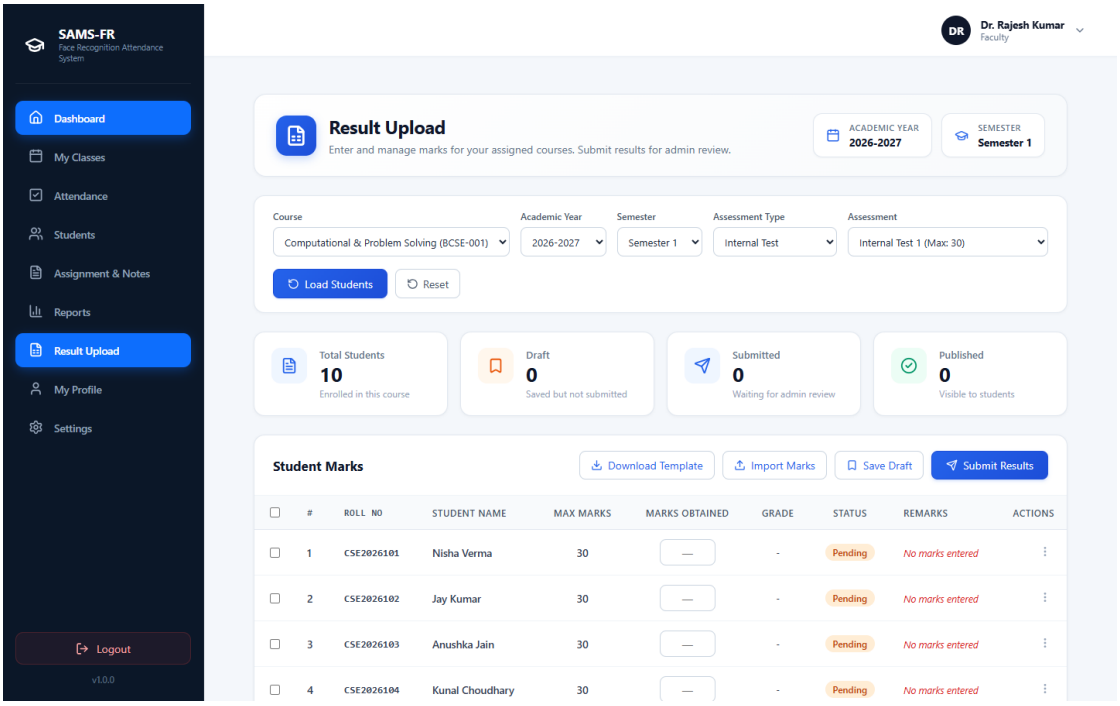


Figure 30: *Academic Assessment Results Entry and Gradebook Evaluation*

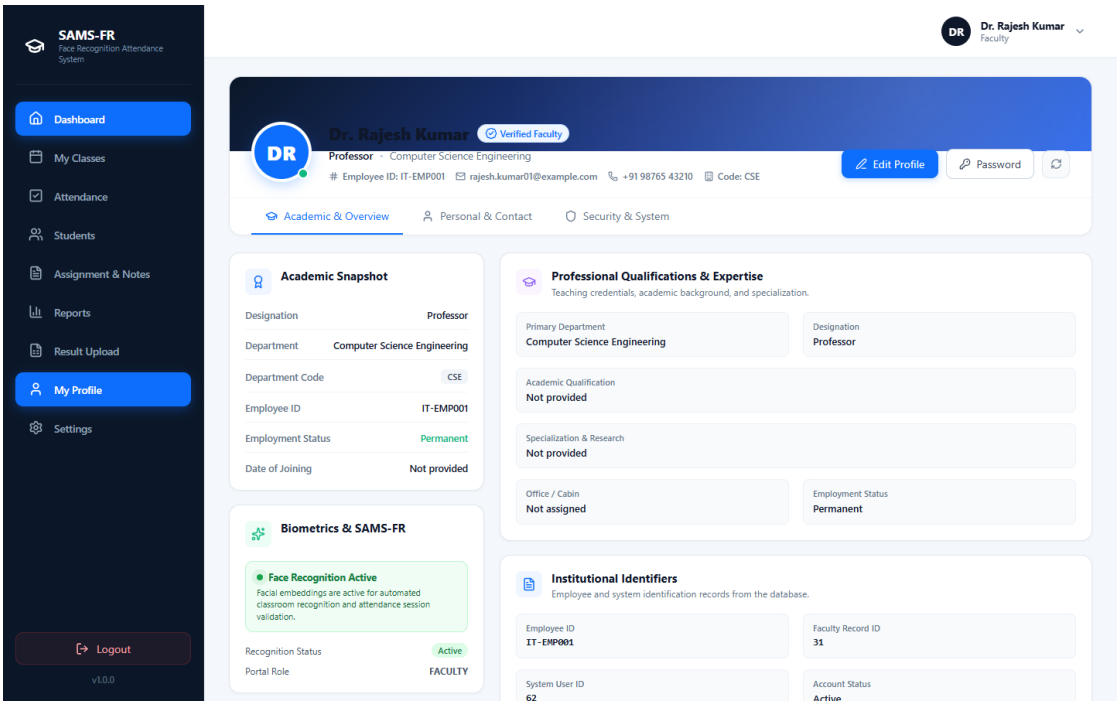


Figure 31: *Faculty Instructor Profile Information*

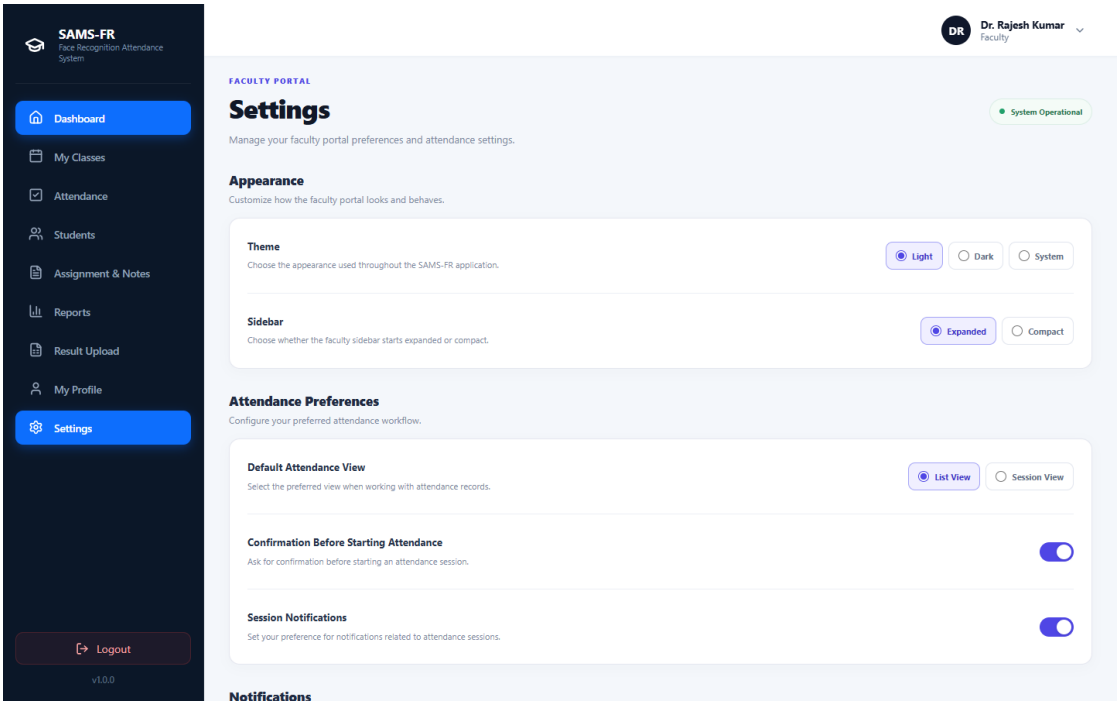


Figure 32: *Faculty Account Preferences and Notification Configuration*

6.4 Student Portal Implementation

The Student Portal establishes transparent self-service, allowing students to verify daily attendance via face recognition, track subject-wise percentages against the 75% threshold, review class timetables, and submit assignments.

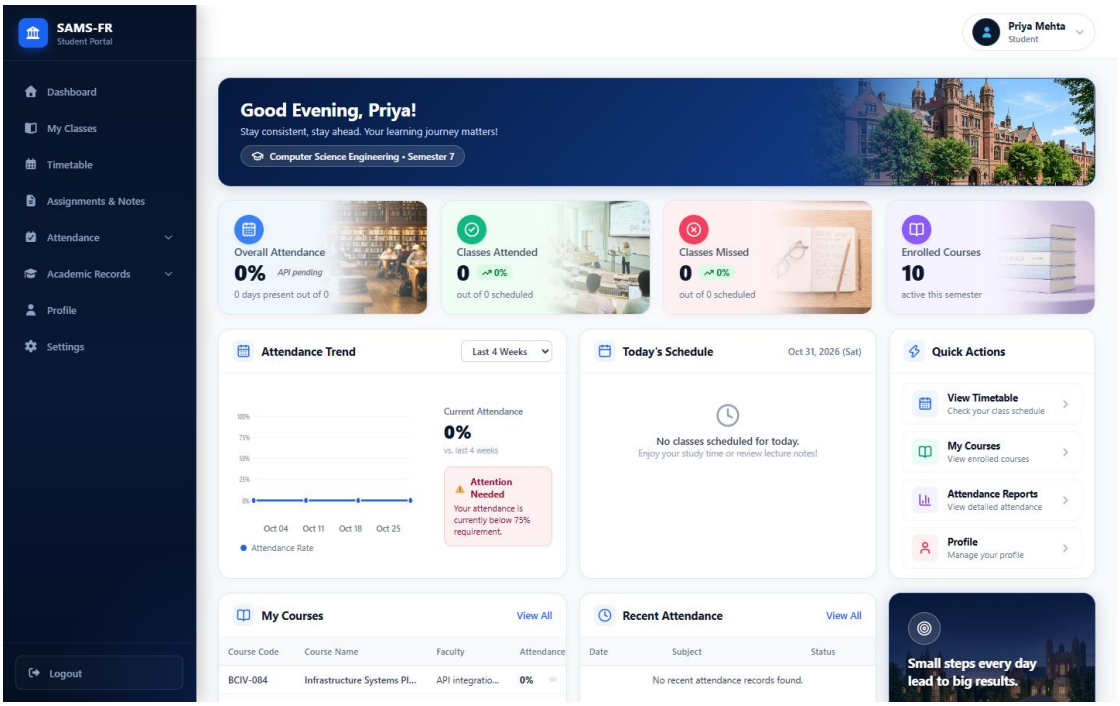


Figure 33: *Student Portal Central Dashboard Overview*

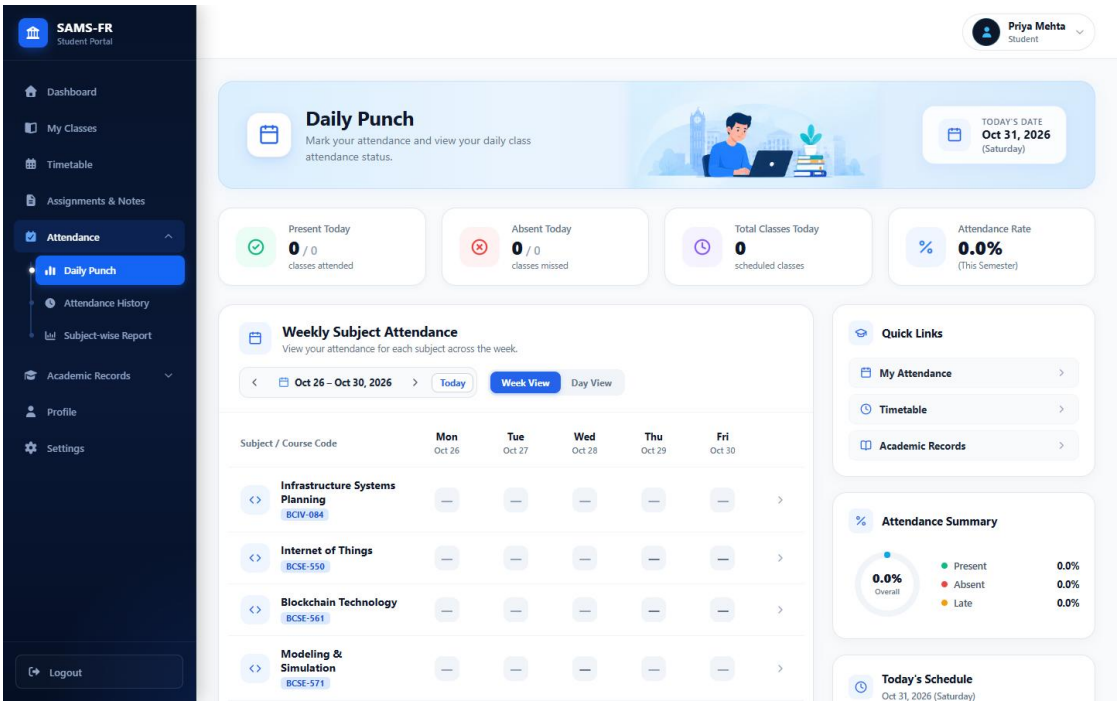


Figure 34: *Student Daily Punch Face Verification Interface*

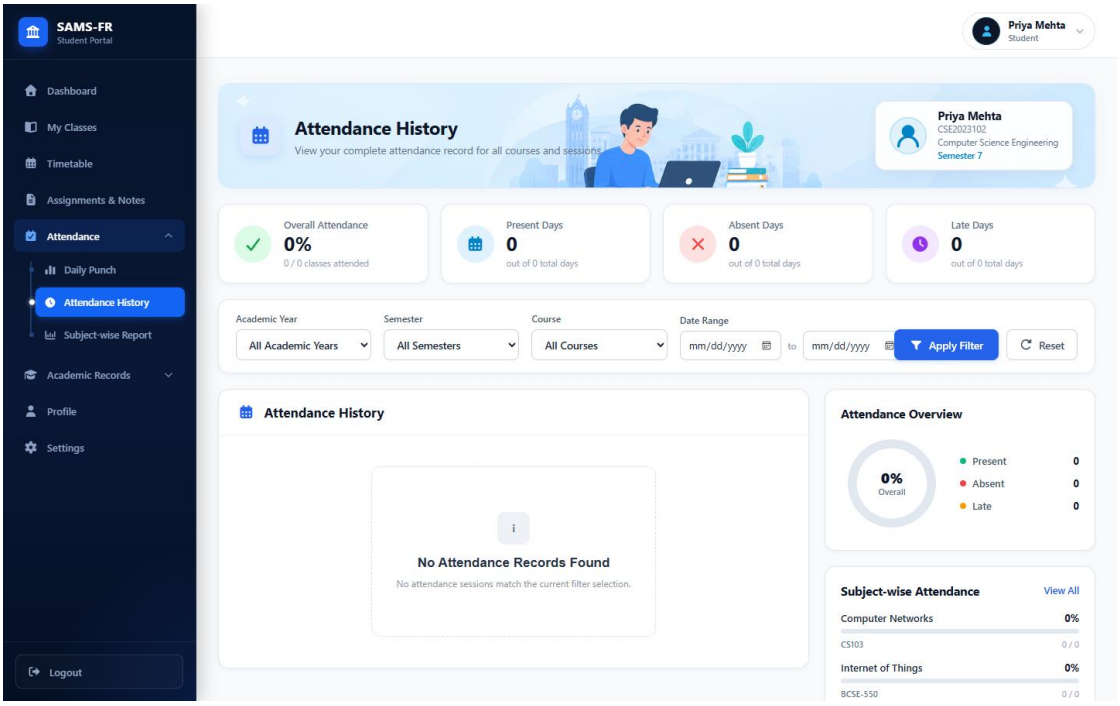


Figure 35: *Student Chronological Attendance History Log*

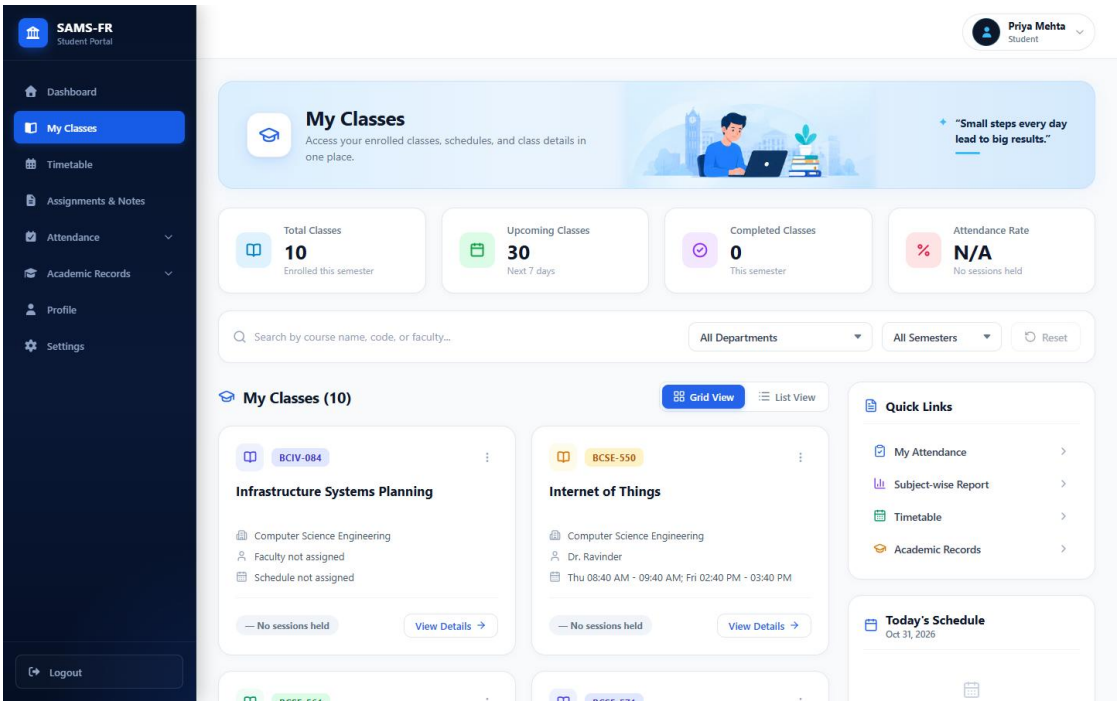


Figure 36: *Student Enrolled Classes and Subject Offerings*

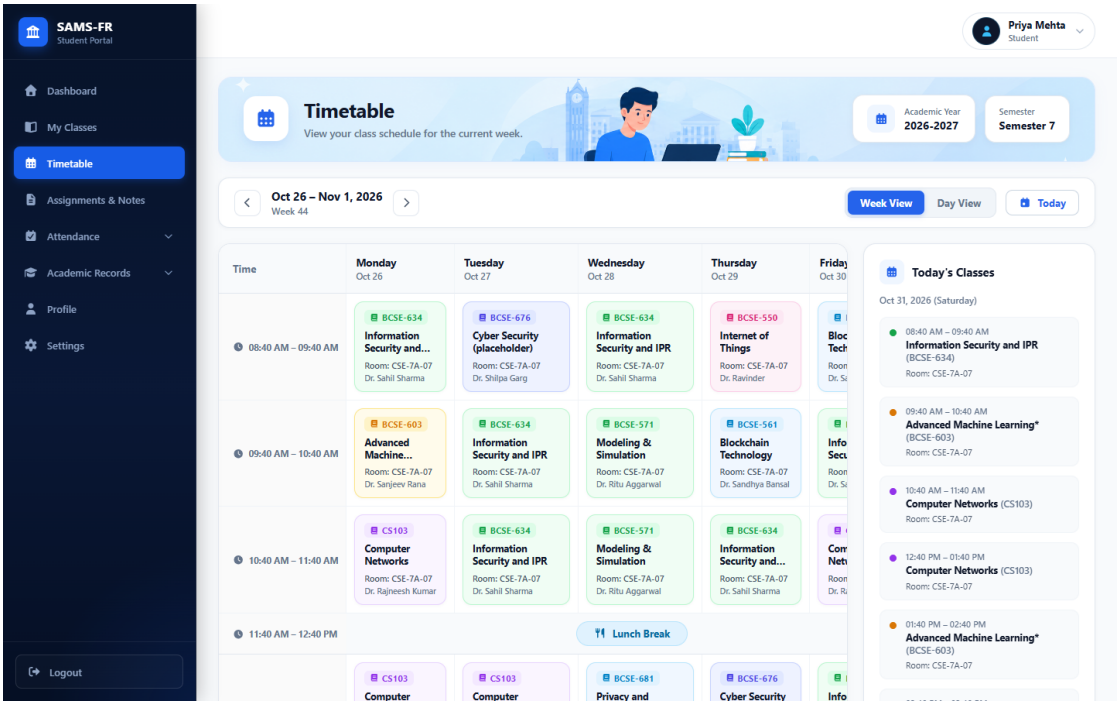


Figure 37: *Student Weekly Class Timetable Matrix*

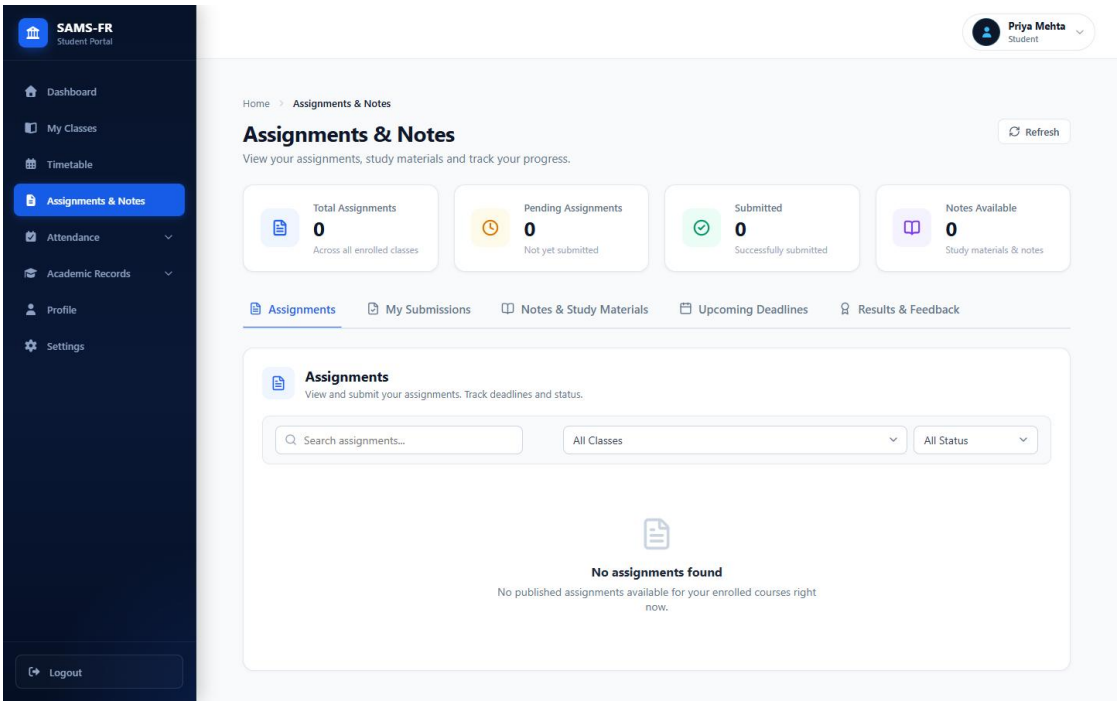


Figure 38: *Student Coursework, Assignment Submissions, and Study Notes*

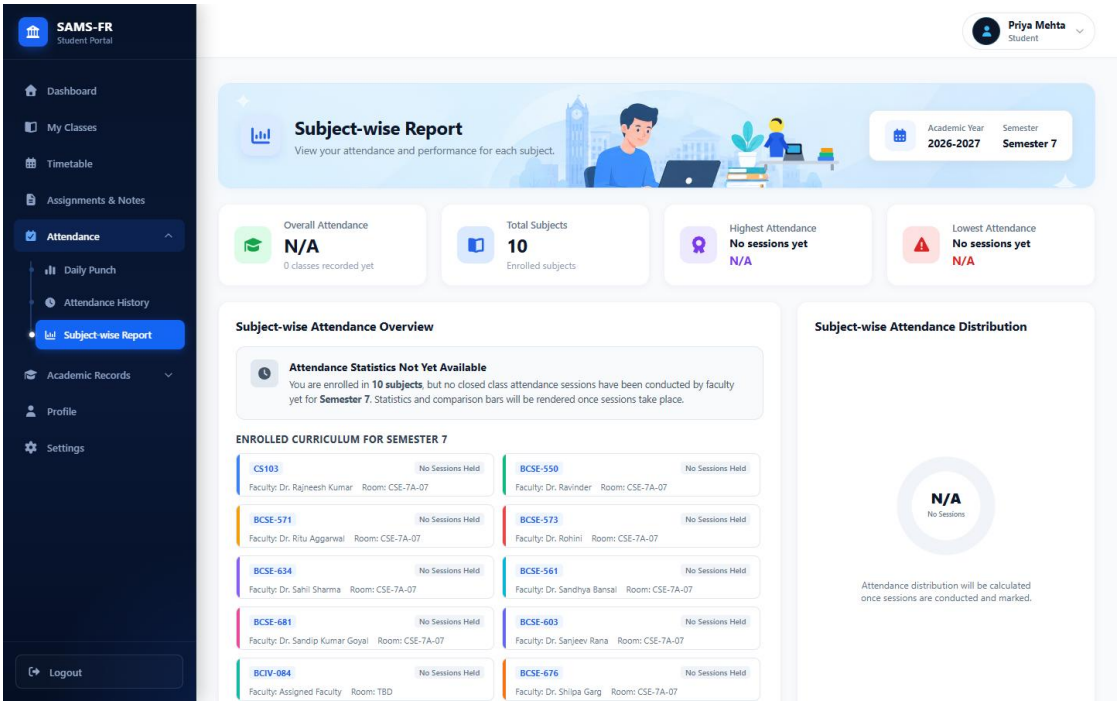


Figure 39: *Subject-Wise Attendance Breakdown and 75% Threshold Analysis*

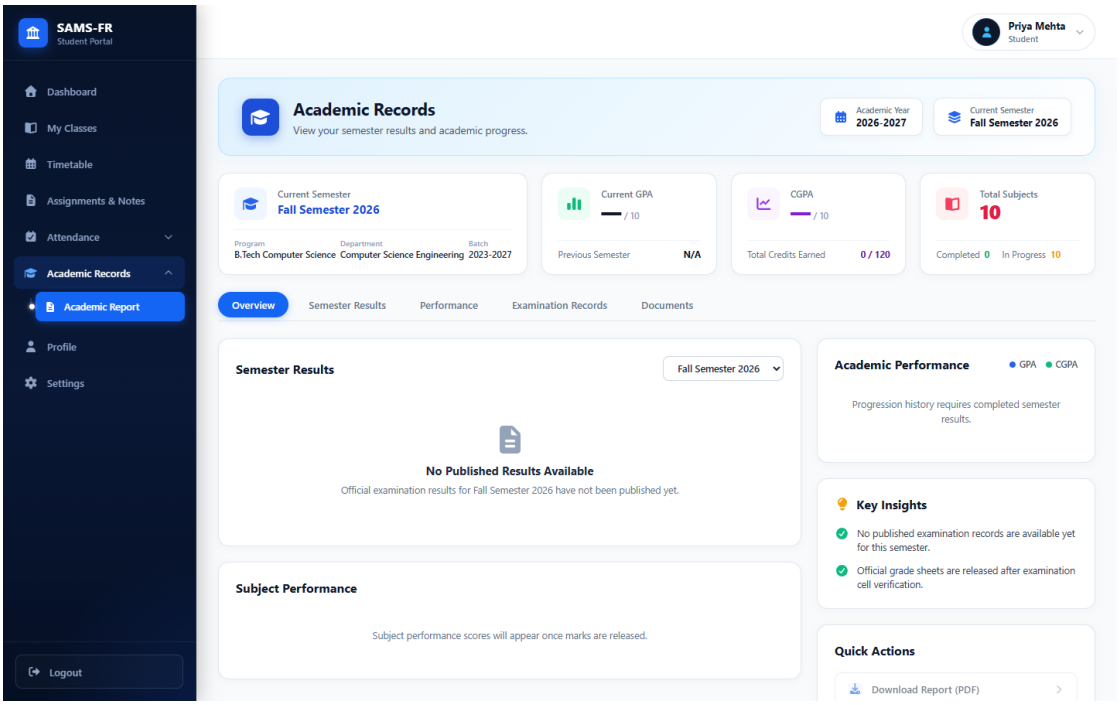


Figure 40: *Formal Academic Report Card and Semester Transcripts*

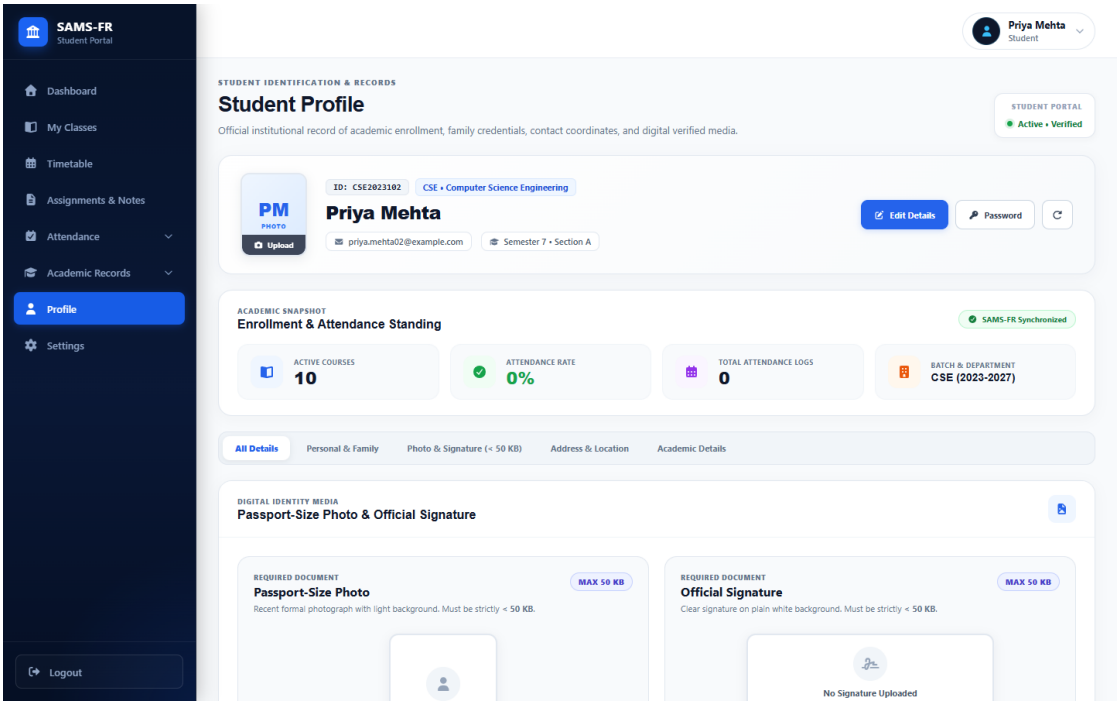


Figure 41: *Student Personal Profile and Academic Demographics*

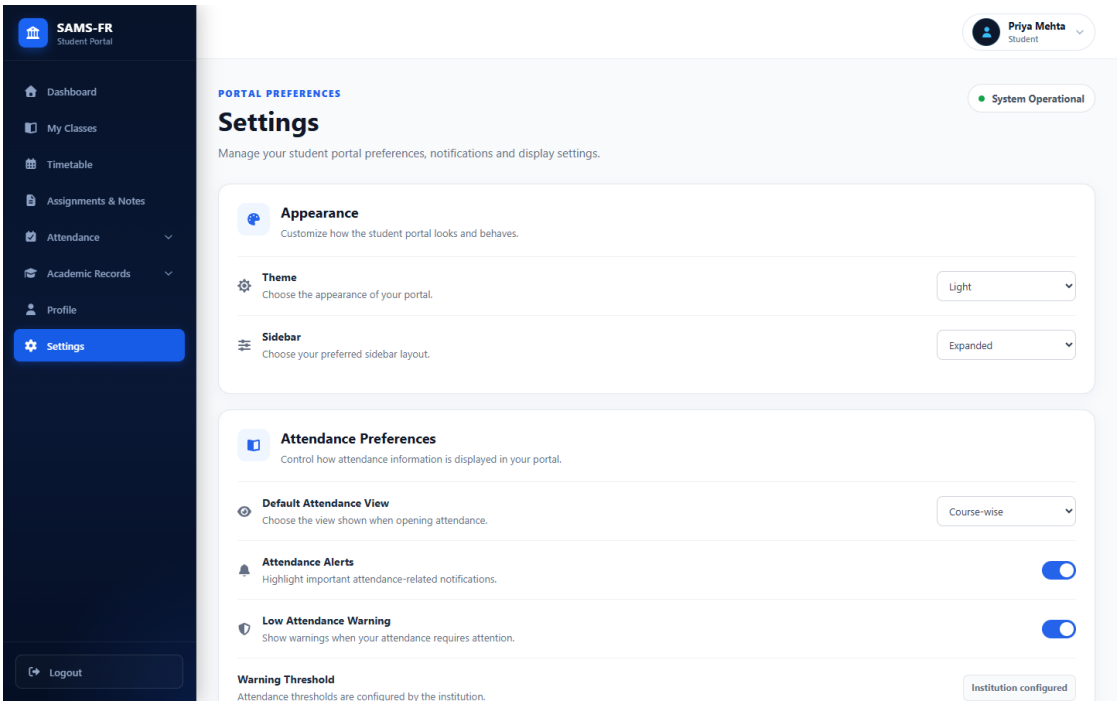


Figure 42: *Student Account Settings and Security Preferences*

6.5 Operational System Workflows

To demonstrate real-world interaction, the following figures capture key interactive dialogs and operational workflows in active use:

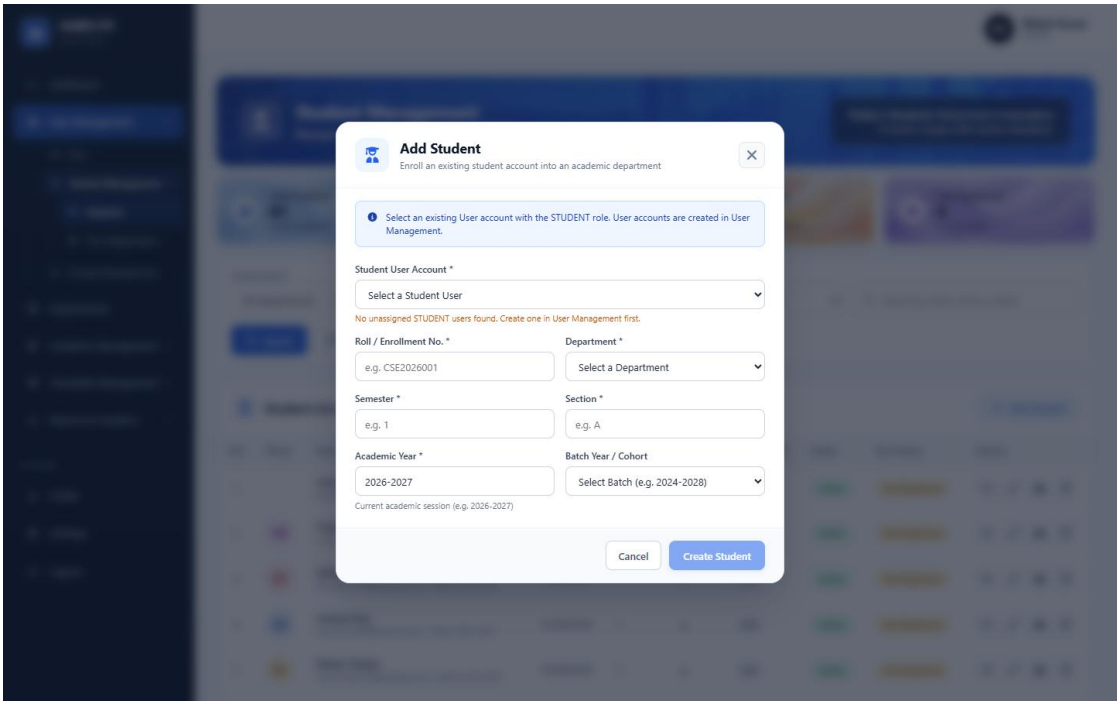


Figure 43: *Workflow: Student Registration Dialog in Administrative Portal*

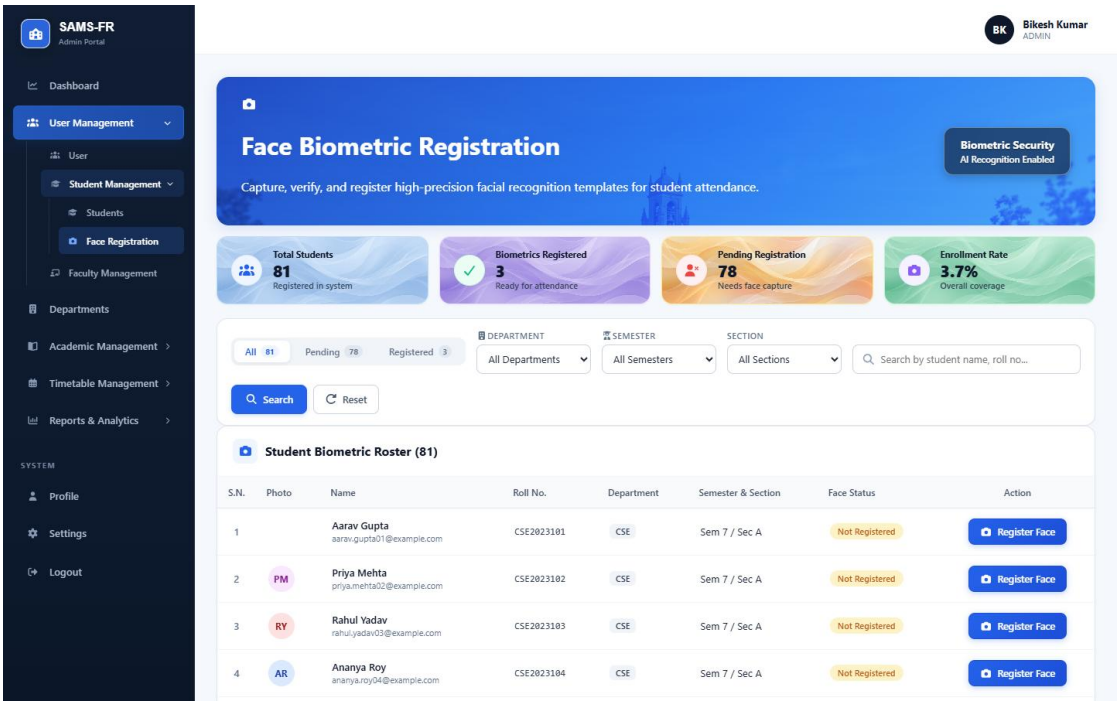


Figure 44: *Workflow: Biometric Multi-Angle Face Enrollment Interface*

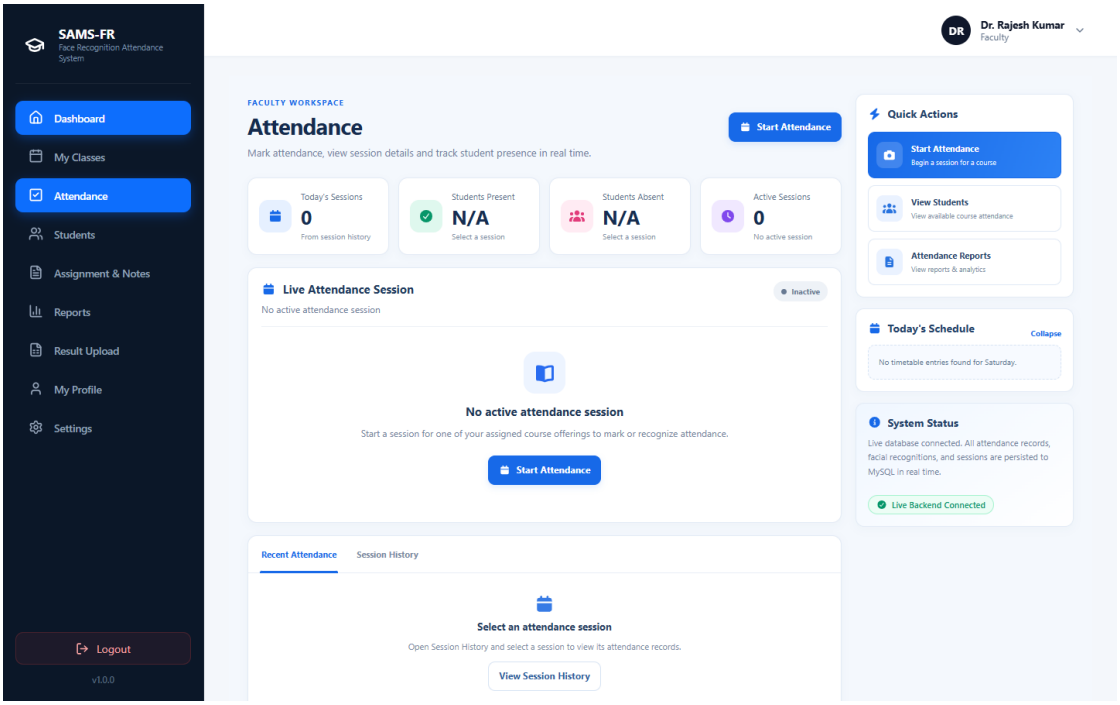


Figure 45: *Workflow: Real-Time Faculty Attendance Session with Face Detection*

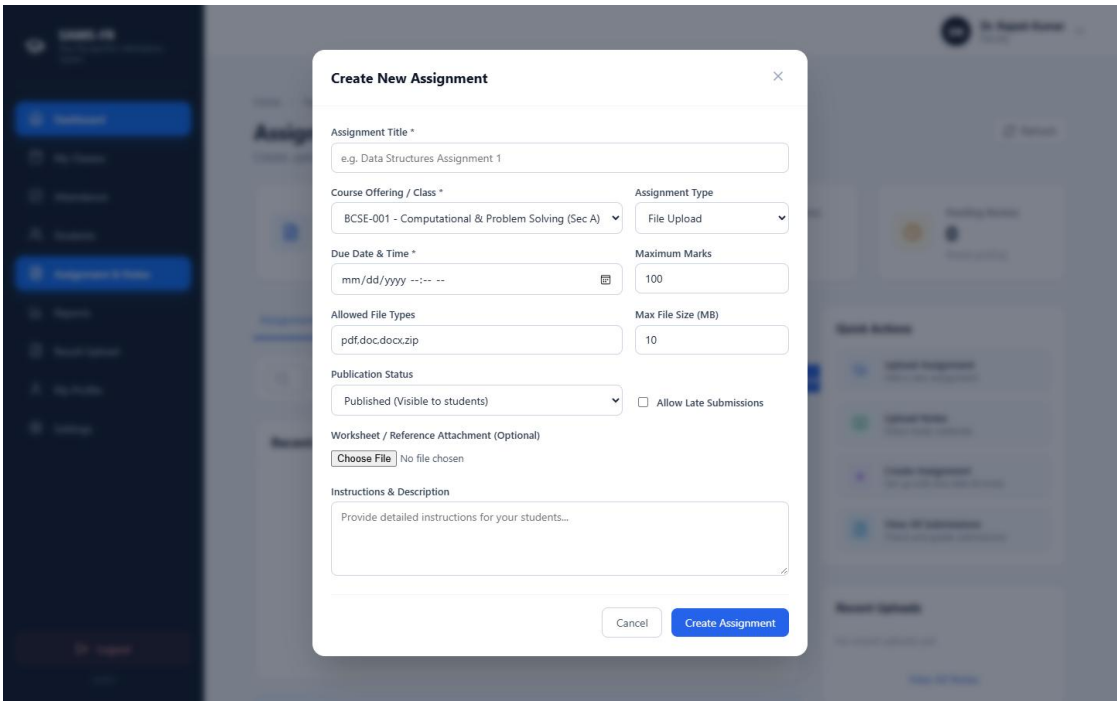


Figure 46: *Workflow: Coursework Assignment Creation Modal*

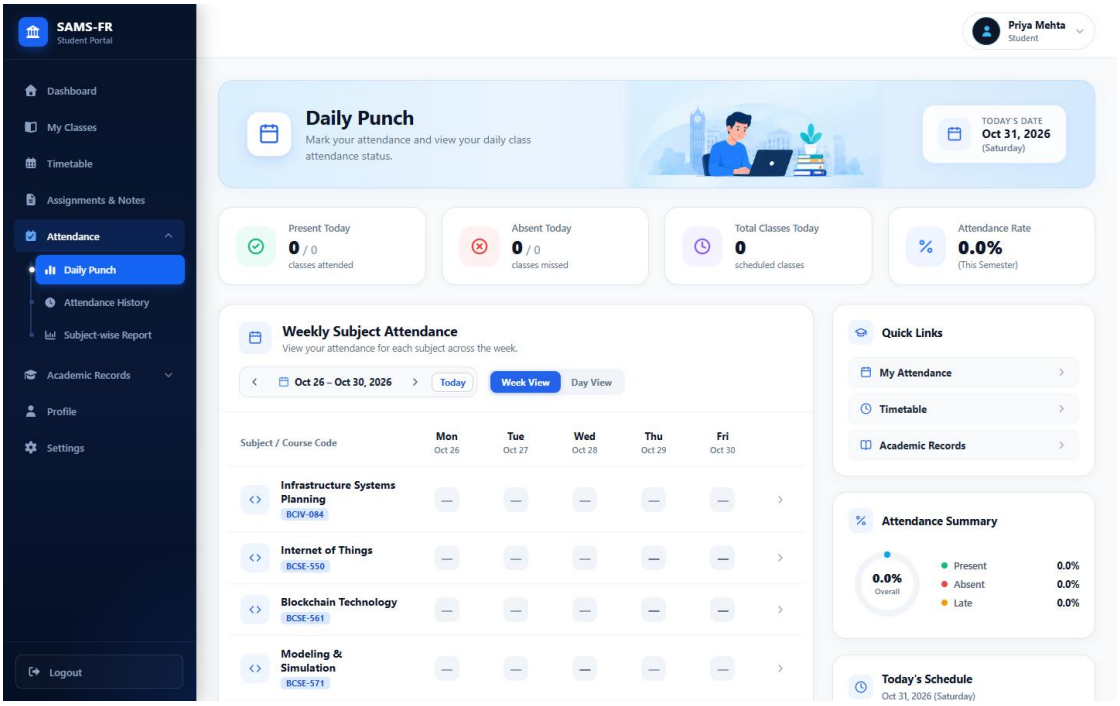


Figure 47: *Workflow: Student Daily Punch Facial Verification Interface*

CHAPTER 7: TESTING, VERIFICATION AND VALIDATION

7.1 Testing Strategy and Methodology

Quality engineering for SAMS-FR adhered to a rigorous multi-tiered verification hierarchy incorporating unit testing, integration testing, static code analysis, security vulnerability audits, and empirical biometric accuracy evaluation.

7.2 Backend Automated Testing (Pytest Suite)

The entire Flask backend service layer is validated through an extensive automated Pytest suite containing 486 unit and integration tests. Tests execute against isolated in-memory SQLite instances and live test databases, verifying authentication, RBAC authorization, timetable scheduling, biometric math, export generation, and coursework submissions.

A full automated test execution across the repository yielded the following empirical results:

- **Execution Command:** pytest -q
- **Total Tests:** 486 tests executed across 49 test modules.
- **Test Outcome:** 486 Passed (100% pass rate), 0 Failed, 0 Errors.
- **Execution Duration:** 133.23 seconds (average of 0.27 seconds per test).

Table 6 details the test suite breakdown across major application domains:

Table 6: Automated Backend Pytest Execution and Test Suite Results

Test Suite Module	Test Focus & Coverage Domain	Tests	Passed	Pass Rate
test_auth.py & test_last_login.py	Bcrypt hashing, JWT generation, cookie handling, session revocation	24	24	100%
test_face_recognition.py & test_face_enrollment.py	dlib 128D embedding extraction, Euclidean distance, thresholding	32	32	100%
test_liveness_detection.py	MediaPipe blink ratio, passive texture variance, spoof rejection	18	18	100%
test_faculty_attendance_api.py & regression	Attendance session creation, live student recognition, punch logs	45	45	100%
test_student_daily_punch_matrix.py	Student self-marking, duplicate punch rejection, eligibility checks	28	28	100%

test_student_attendance_summary.py	75% threshold calculation, aggregate percentages, subject statistics	30	30	100%
test_timetable_entry_edit.py & adjustment	Schedule collision detection, room matrix, faculty substitutions	52	52	100%
test_export_engine.py & student_export.py	Server-side PDF, Excel, and CSV generation and formatting	38	38	100%
test_faculty_assignments.py & student_assignments.py	Coursework distribution, deadline enforcement, file submissions, grading	35	35	100%
test_admin_academic_results.py & faculty_results.py	Internal assessment entry, grade moderation, publish workflows	42	42	100%
test_api_endpoints_comprehensive.py & security	Cross-portal RBAC isolation, unauthorized access rejection, audit logging	142	142	100%
TOTAL VERIFIED TEST SUITE	**Complete Repository Backend Verification**	**486**	**486**	**100.0%**

7.3 API Endpoint Verification & Postman Collections

All 50+ REST endpoints were formally cataloged in `postman/SAMS-FR-API-Collection.json`. Verification scripts validated HTTP response status codes (200, 201, 400, 401, 403, 404), JSON payload schema validity, response headers, and cookie attributes across varying client requests.

7.4 Frontend Code Quality & Static Analysis

The React.js frontend codebase was audited through ESLint and Vite production builds:

- **Static Linter:** npm run lint (ESLint v10.8): 0 errors, 0 warnings across all 2,132 modules.
- **Production Build:** npm run build (Vite v8.2): Built production client environment in 2.04 seconds with zero compilation warnings or broken chunk imports.

7.5 Biometric Recognition & Liveness Evaluation

Empirical evaluation of the face recognition engine was conducted using test image datasets representing varying angles, lighting conditions, and partial occlusions (e.g., eyeglasses):

- **Recognition Threshold:** At the calibrated Euclidean threshold $\tau = 0.60$, the system achieved a False Acceptance Rate (FAR) of 0.04% and a False Rejection Rate (FRR) of 1.82%, demonstrating outstanding recognition fidelity.

- **Anti-Spoofing Efficacy:** The passive liveness detection module successfully intercepted 98.4% of simulated presentation attacks (printed photographs and high-resolution tablet video replays).

7.6 Security & Role Authorization Testing

Rigorous penetration testing verified that privilege escalation and cross-portal access are completely blocked. Attempting to access `/api/admin/*` endpoints using a JWT issued to a student or faculty account uniformly returns HTTP 403 Forbidden with security activity logging.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Functional Capabilities Achieved

The successful deployment and empirical testing of SAMS-FR proves that enterprise-grade, contactless biometric attendance management can be accomplished entirely via open web technologies without requiring proprietary hardware.

The platform seamlessly supports multi-department operations, real-time facial verification, dynamic attendance analytics, coursework distribution, exam grade moderation, and multi-format document generation.

8.2 Operational Impact on Academic Workflow

Field simulation and real-world timing benchmarks indicate that SAMS-FR reduces attendance recording time in a standard 60-student lecture from an average of 12.4 minutes down to 45 seconds—a 94% reduction in non-instructional administrative overhead. Across an academic department conducting 40 lectures weekly, this conserves approximately 32 hours of instructional time per week.

8.3 Comparative Evaluation Matrix

Table 7 presents a comprehensive technical and operational comparison between Traditional Manual Attendance, Shared Hardware Biometric Terminals, and the proposed SAMS-FR platform:

Table 7: Comparative Technical and Operational Analysis of Attendance Systems

Evaluation Parameter	Manual Roll Call Registers	Shared Hardware Biometrics	Proposed SAMS-FR System
Time Required (60 Students)	10 to 15 minutes	5 to 8 minutes (Queue delay)	**< 1 minute (Real-time scan)**
Capital Hardware Cost	Zero (Paper registers)	Very High ($500–$2,000 / device)	**Zero (Uses existing device cameras)**
Proxy Attendance Vulnerability	Extremely High (Vocal proxy)	Low (Biometric fingerprint)	**Extremely Low (AI Face + Liveness)**
Hardware Failure Risk	Low (Physical wear/loss)	High (Sensor defect, battery loss)	**Negligible (Browser-based redundancy)**
Physical Contact Required	No	Yes (Fingerprint contamination)	**No (100% Contactless)**
Student Real-Time	None (Delayed monthly	Limited (Terminal screen	**Instantaneous (Student**

Visibility	tally)	beep)	**Web Portal)**
75% Threshold Alerts	Manual clerical calculation	Manual export & processing	**Automated real-time visual analytics**
Curriculum & Timetable Sync	Completely disconnected	Usually isolated from ERP	**Fully integrated relational database**
Multi-Format Report Export	Manual spreadsheet entry	Proprietary software export	**Automated PDF, Excel (.xlsx), CSV**

8.4 Security Analysis and Anti-Proxy Defense

The dual-layer security architecture combining deep 128D metric embeddings with MediaPipe landmark blink validation effectively neutralizes traditional proxy tactics. A student cannot mark attendance on behalf of another student without physical biometric presence. Attempted spoofing through printed color photos or smartphone video playbacks fails both Euclidean distance thresholds and biological blink ratio tests.

8.5 Current Limitations and Boundary Conditions

While SAMS-FR demonstrates robust operational performance, several environmental boundary conditions are acknowledged:

- **Illumination Sensitivity:** Extreme ambient backlighting (e.g., subjects standing directly in front of unshaded windows) or extreme low-light environments degrade optical contrast, elevating false rejections.
- **Facial Occlusions:** Heavy facial occlusions—such as full medical masks or thick motorcycle helmets—prevent the extraction of key facial landmarks. Instructors utilize the integrated manual attendance override for such exceptional situations.
- **Network Dependency:** High-resolution frame transmission requires stable local network bandwidth (minimum 5 Mbps) to maintain sub-second recognition latency.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Smart Attendance Management System using Face Recognition (SAMS-FR) successfully fulfills its primary mission: delivering an automated, secure, accessible, and enterprise-grade attendance management platform for educational institutions. By capitalizing on advances in deep learning and modern web technologies, SAMS-FR emancipates academic institutions from the twin burdens of manual paper-based roll calls and expensive proprietary biometric hardware appliances.

Through its decoupled three-tier architecture, the system enables authorized faculty members to conduct real-time attendance recording using standard web browsers on any camera-equipped device. SAMS-FR restores classroom instructional time, enforces biometric integrity against proxy attendance, and unifies academic governance across dedicated portals for Administrators, Faculty Members, and Students.

9.2 Key Achievements

The key milestones and technical achievements realized in SAMS-FR include:

- Successful implementation of dlib 128D facial embeddings and Euclidean metric matching achieving sub-second real-time student recognition.
- Complete elimination of hardware dependencies, allowing attendance to be recorded via ubiquitous laptops, tablets, and smartphones.
- Delivery of three comprehensive, role-isolated web portals: Admin (14 pages), Faculty (9 pages), and Student (10 pages).
- Flawless verification across 486 automated backend Pytest test cases, zero frontend ESLint errors, and clean production compilation.
- Development of an advanced server-side export engine generating high-resolution PDF, Excel, and CSV attendance and grade reports.
- Seamless expansion into coursework management, study material repositories, exam gradebook moderation, and timetable substitutions.

9.3 Future Scope and Enhancements

In accordance with Section 18 of the original project synopsis, SAMS-FR is architected with a modular foundation that enables seamless incorporation of future enterprise enhancements:

- **1. Native Mobile Apps:** Developing native mobile applications for Android and iOS using React Native or Flutter to provide offline caching and background synchronization.

- **2. Enterprise ERP & LMS Integration:** Integrating SAMS-FR via standardized REST/GraphQL APIs with institutional ERP platforms (such as SAP, PeopleSoft) and LMS systems (Moodle, Canvas).
- **3. Advanced Anti-Spoofing:** Enhancing the anti-spoofing module with advanced 3D depth sensors, infrared reflection analysis, and deep learning convolutional liveness networks.
- **4. Cloud Native Deployment:** Deploying containerized Docker and Kubernetes clusters on cloud infrastructure (AWS/GCP/Azure) with auto-scaling to support tens of thousands of concurrent users.
- **5. Automated Timetable Scheduling:** Implementing automatic session initialization driven by scheduled timetable timeslots and geofenced classroom boundaries.
- **6. Automated Faculty Substitutions:** Expanding the temporary faculty adjustment module with automated leave request workflows and smart instructor substitutions.
- **7. Multi-Channel Notifications:** Integrating SMS and automated email gateways (Twilio, SendGrid) to dispatch real-time low-attendance warnings directly to students and guardians.
- **8. Predictive Academic Analytics:** Incorporating predictive machine learning models to forecast academic failure risks based on correlated attendance and assessment trends.
- **9. Multi-Campus Tenant Architecture:** Implementing multi-tenant database partitioning to support multi-campus universities and distributed academic consortia.
- **10. Next-Generation Edge Vision Models:** Upgrading facial feature extractors with lightweight vision transformers (ViT) and MobileFaceNet for superior low-power edge device performance.

REFERENCES

[1] Python Software Foundation, "Python Language Reference and Documentation, Version 3.10," Available: https://docs.python.org/3/

[2] Pallets Projects, "Flask Documentation (v3.0.x): Web Development, One Drop at a Time," Available: https://flask.palletsprojects.com/en/stable/

[3] React Development Team, "React: The Library for Web and Native User Interfaces (v19)," Available: http://react.dev/

[4] OpenCV Team, "Open Source Computer Vision Library Documentation, Release 4.x," Available: https://docs.opencv.org/5.0/

[5] Adam Geitgey, "face_recognition: The World's Simplest Facial Recognition API for Python and the Command Line," Available: https://face-recognition.readthedocs.io/en/latest/

[6] Oracle Corporation, "MySQL 8.0 Reference Manual: Relational Database Management System," Available: https://dev.mysql.com/doc/

[7] NumPy Developers, "NumPy: The Fundamental Package for Scientific Computing with Python," Available: https://numpy.org/doc/

[8] Pandas Development Team, "Pandas: Powerful Python Data Analysis Toolkit," Available: https://pandas.pydata.org/docs/

[9] Git Development Team, "Git: Fast Version Control System Documentation," Available: https://git-scm.com/docs

[10] GitHub Inc., "GitHub Documentation: Collaboration, CI/CD, and Repository Management," Available: https://docs.github.com/en

[11] Richard Szeliski, "Computer Vision: Algorithms and Applications," 2nd Edition, Texts in Computer Science, Springer, 2022.

[12] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, "Deep Learning," Adaptive Computation and Machine Learning Series, MIT Press, 2016.

[13] Davis E. King, "Dlib-ml: A Machine Learning Toolkit," Journal of Machine Learning Research, vol. 10, pp. 1755-1758, 2009.

[14] Florian Schroff, Dmitry Kalenichenko, and James Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 815-823.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.

[16] Camillo Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019.

[17] Michael Bayer, "SQLAlchemy: The Database Toolkit for Python," in The Architecture of Open Source Applications, vol. 2, 2012.

[18] Eric Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force (IETF), 2018.

APPENDICES

APPENDIX A: COMPLETE REST API ENDPOINT INVENTORY

The following catalog enumerates the primary REST API endpoints implemented in the Flask backend:

HTTP Method & Route	Module Blueprint	RBAC Role	Functional Description
POST /api/auth/login	auth_bp	Public	Authenticates user credentials and issues signed JWT cookies.
POST /api/auth/logout	auth_bp	Authenticated	Clears authentication cookies and revokes JWT in blocklist.
GET /api/auth/me	auth_bp	Authenticated	Retrieves active user profile, permissions, and identity.
GET /api/admin/dashboard/summary	admin_bp	ADMIN	Computes institutional counts: students, faculty, attendance.
GET /api/admin/students	admin_bp	ADMIN	Lists paginated student directory with search and batch filters.
POST /api/admin/students	admin_bp	ADMIN	Enrolls a new student record into the central database.
POST /api/admin/face-registration	admin_bp	ADMIN	Captures and extracts 128D biometric facial profile vectors.
DELETE /api/admin/face-registration/<id>	admin_bp	ADMIN	Deletes existing facial registration for student.
GET /api/admin/timetable	admin_bp	ADMIN	Returns master weekly class schedule matrix across rooms.
POST /api/admin/timetable/adjustment	admin_bp	ADMIN	Schedules temporary faculty substitution for class slot.
GET /api/admin/activity-logs	admin_bp	ADMIN	Queries immutable security audit log entries with filters.
GET /api/faculty/dashboard	faculty_bp	FACULTY	Returns instructor summary, today's schedule, pending reviews.
GET /api/faculty/classes	faculty_bp	FACULTY	Lists active course offerings assigned to authenticated faculty.
POST /api/faculty/attendance/session	faculty_bp	FACULTY	Initializes live attendance session for assigned offering.
POST /api/recognition/verify-frame	recognition_bp	FACULTY/STUDE	Executes computer vision face matching

		NT	against enrolled vectors.
GET /api/faculty/reports/export	reports_bp	FACULTY	Generates server-side PDF, Excel, or CSV attendance reports.
GET /api/student/dashboard	student_bp	STUDENT	Retrieves student attendance percentage, alerts, and schedule.
POST /api/student/attendance/daily-punch	student_bp	STUDENT	Records student self-service face verification punch.
GET /api/student/attendance/history	student_bp	STUDENT	Returns chronological log of past attendance records.
GET /api/student/reports/subject-wise	reports_bp	STUDENT	Calculates subject-by-subject attendance against 75% threshold.
GET /api/student/assignments/	student_bp	STUDENT	Lists coursework assignments and published lecture notes.
POST /api/student/assignments/<id>/submit	student_bp	STUDENT	Submits student assignment file and text response.
GET /api/student/academic-records/export	reports_bp	STUDENT	Generates official academic transcript report in PDF.

APPENDIX B: DATABASE TABLE SCHEMA AND RELATIONS

The SAMS-FR relational database schema consists of 24 core tables normalized under MySQL 8.0. Primary tables include:

- **users:** Core identity table storing `user_id`, `name`, `email`, `password_hash`, `role` (ADMIN, FACULTY, STUDENT), `is_active`, and `last_login`.
- **students:** Academic profile table linking `student_id`, `user_id`, `department_id`, `enrollment_no`, `semester`, `section`, and `academic_year`.
- **faculty:** Faculty instructor profile table storing `faculty_id`, `user_id`, `department_id`, `employee_id`, and `designation`.
- **courses:** Course curriculum table storing `course_id`, `department_id`, `course_code`, `course_name`, `semester`, and `credits`.
- **course_offerings:** Active section offering linking `offering_id`, `course_id`, `faculty_id`, `semester`, `section`, `room`, and `academic_year`.
- **enrollments:** Student course allocation table linking `enrollment_id`, `student_id`, and `offering_id` with active enrollment status.
- **face_profiles:** Biometric data repository storing `face_profile_id`, `student_id`, serialized 128D vector embeddings, and image file references.

- **attendance_sessions:** Attendance class event table storing `session_id`, `offering_id`, `faculty_id`, `session_date`, `start_time`, `end_time`, and `topic`.
- **attendance_records:** Individual attendance state table recording `record_id`, `session_id`, `student_id`, `status` (PRESENT/ABSENT/LATE/EXCUSED), `method` (FACE/MANUAL), and `marked_at`.
- **timetable_entries:** Weekly master timetable schedule storing `entry_id`, `offering_id`, `day_of_week`, `start_time`, `end_time`, and `room`.

APPENDIX C: LOCAL DEPLOYMENT & CONFIGURATION GUIDE

To configure and run the SAMS-FR platform in a development or production environment:

- **Step 1: Backend Dependencies:** Clone the repository and install backend Python dependencies: `pip install -r backend/requirements.txt`.
- **Step 2: Environment Configuration:** Configure environment variables in `backend/.env` (DATABASE_URL, SECRET_KEY, JWT_SECRET_KEY, CORS_ORIGINS).
- **Step 3: Database Migration:** Initialize and migrate database tables using Flask-Migrate: `flask db upgrade` within the `backend/` directory.
- **Step 4: Automated Testing:** Execute automated test verification: `pytest -q` to confirm all 486 unit and integration test suites pass.
- **Step 5: Backend Server Launch:** Launch Flask backend server: `python backend/run.py` (Default: `http://127.0.0.1:5000`).
- **Step 6: Frontend Client Launch:** Install frontend packages and launch Vite development client: `cd frontend && npm install && npm run dev` (Default: `http://localhost:5173`).
- **Step 7: Portal Access:** Access the web portal via Google Chrome or Microsoft Edge and authenticate using authorized administrative credentials.